FE

SERVICE MNC · INFOSYS · TCS · WIPRO · COGNIZANT · ACCENTURE · CAPGEMINI · HCLTECH · LTIMINDTREE

Frontend Engineer in a Service MNC

> ఈ ఉద్యోగం నిజంగా ఏమిటి – levels, pay, tech stack, రోజువారీ పని, hiring funnel, మరియు లోపలికి వచ్చాక ఎలా ఎదగాలి.

ఇది interview prep doc కాదు – ఇది **role** doc. Interview rounds కోసం **Infosys_Interview_Prep_Telugu.md**; React/JS లోతు కోసం **React_Telugu.md**, **JavaScript_Telugu.md**. ఇది ఆ రెండిటి చుట్టూ ఉన్న **context** – ఏ ఉద్యోగానికి తయారవుతున్నావో అది.

తెలుగు + ENGLISH

YASWANTH · REFERENCE

విషయ సూచిక (Table of Contents)

Part 0 – ఈ ఉద్యోగం నిజంగా ఏమిటి

1. Service company vs Product company – business model నీ ఉద్యోగాన్ని ఎలా నిర్ణయిస్తుంది
2. Billability, bench, allocation, utilization – నీ career ని నడిపే నాలుగు మాటలు
3. Account, project, pod – ఎవరు ఎవరు
4. Frontend engineer రోజూ ఏం చేస్తాడు – JD చెప్పేది vs నిజం

Part 1 – Levels, Designations, Pay

5. ఏడు కంపెనీల level ladder – ఒక పోలిక పట్టిక
6. నీ experience కి ఏ level వస్తుంది
7. Lateral hire vs internal promotion – laterals grades ఎందుకు దూకుతారు
8. Offer lo నీ level ని నిజంగా ఏది నిర్ణయిస్తుంది
9. Level code ఎందుకు పట్టించుకోవాలి
10. CTC structure – fixed, variable, retention; ఏది నిజమైన డబ్బు

Part 2 – నిజమైన Tech Stack

11. React vs Angular – Indian services lo వాస్తవం
12. ఒక typical enterprise React project ఎలా ఉంటుంది
13. Legacy & migration projects – jQuery, AngularJS, Razor
14. Micro-frontends & Module Federation – ఎప్పుడు నిజం, ఎప్పుడు resume-driven
15. Design system & component library పని
16. Accessibility (WCAG 2.2 AA) – ఇది contract, option కాదు
17. Java/Spring backend – frontend dev కి ఎంత తెలియాలి
18. Testing, CI/CD, release process
19. Service company lo నువ్వు చేయనివి

Part 3 – ఒక రోజు, ఒక sprint, ఒక సంవత్సరం

20. రోజువారీ – standup, tickets, PR review, timesheet
21. Sprint ceremonies – planning, grooming, demo, retro
22. SAFe / PI planning – పెద్ద accounts lo
23. Client తో మాట్లాడటం – status call, demo, defect triage, UAT
24. Onshore-offshore model – timezone, handover, "overlap hours"
25. Production support – L2/L3 rotation, RCA
26. Appraisal cycle – rating నిజంగా ఎలా వస్తుంది

Part 4 – లోపలికి రావడం (Hiring)

27. Requisition ఎక్కడ నుంచి వస్తుంది – ఇది నీ వ్యూహాన్ని మారుస్తుంది

28. Apply ఎలా – portal, Naukri, referral, vendor
29. Recruiter screen – వాళ్ళు ఏం చూస్తున్నారు
30. కంపెనీల వారీగా rounds – ఒక పోలిక
31. Technical round – 2–5 years కి నిజంగా ఏం అడుగుతారు
32. JavaScript bank – నిజమైన eliminator
33. React bank
34. HTML / CSS / Accessibility bank
35. చిన్న coding task – machine coding round
36. Frontend system design – SSE స్థాయిలో
37. Managerial round – project ownership కథ
38. HR round – negotiation, notice period, documents

Part 5 – లోపలికి వచ్చాక ఎదగడం

39. మొదటి 90 రోజులు – కొత్త account lo
40. Rating తెచ్చేది vs promotion తెచ్చేది – ఇవి వేరు
41. Certifications & internal learning platforms
42. Onsite opportunity – నిజంగా ఎలా నిర్ణయిస్తారు
43. IC track vs Lead track – ఎప్పుడు ఏది
44. ఎప్పుడు మారాలి – project change vs company change
45. Exit path – services నుంచి GCC / product కి

Part 6 – Red Flags & నిజాలు

46. Requisition / project lo red flags
47. Support project vs development project
48. Stack stagnation – ఎలా తప్పించుకోవాలి
49. AI shift – ఏది commodity అయ్యింది, ఏది విలువ పెరిగింది

ముగింపు – 30 రోజుల ప్రణాళిక

ఈ doc లోని సంఖ్యల గురించి ఒక నిజాయితీ గమనిక. Level ladders మరియు CTC bands ని ఏ కంపెనీ కూడా బహిరంగంగా ప్రచురించదు. ఇక్కడ ఉన్నవి Levels.fyi, Glassdoor, Fishbowl, Grapevine, Quora లాంటి చోట్ల ఉద్యోగులు స్వయంగా report చేసిన డేటా నుంచి – అంటే **సూచికలు, హామీలు కాదు**. Location, skill, negotiation, ఆ సమయంలో ఉన్న demand – ఇవన్నీ ఒక్కో offer ని ±30% కదిలిస్తాయి. ఈ సంఖ్యలని *సంభాషణ మొదలుపెట్టే స్థానం* గా వాడు, *హక్కు* గా కాదు.

Part 0 – ఈ ఉద్యోగం నిజంగా ఏమిటి

1. Service company vs Product company – business model నీ ఉద్యోగాన్ని ఎలా నిర్ణయిస్తుంది

వివరణ

ఇది ఈ doc మొత్తంలో **అత్యంత ముఖ్యమైన section**. మిగతాదంతా దీని నుంచే వస్తుంది.

ఒక product company (Google, Swiggy, Razorpay) **తన సొంత product** ని అమ్ముతుంది. వాళ్ళ డబ్బు product నుంచి వస్తుంది. కాబట్టి వాళ్ళకి కావలసింది – ఆ product ని మెరుగుపరిచే engineers. ఒక engineer product ని 2% మెరుగుపరిస్తే, అది కోట్ల రూపాయలు కావొచ్చు. అందుకే వాళ్ళు **తక్కువ మందిని, ఎక్కువ జాగ్రత్తగా** తీసుకుంటారు, ఎక్కువ జీతం ఇస్తారు, మరియు interview lo "నువ్వు ఎంత తెలివైనవాడివి?" అని చూస్తారు.

ఒక service company (Infosys, TCS, Wipro, Cognizant, Accenture) **తన సొంత product ని అమ్మదు** – అది **మనుషుల సమయాన్ని** అమ్ముతుంది. ఒక American bank కి software కావాలి; ఆ bank తన సొంత team ని పెంచుకునే బదులు Infosys తో contract చేసుకుంటుంది. Infosys 40 మందిని ఆ project కి కేటాయిస్తుంది, మరియు **ప్రతి మనిషికీ గంటకి ఇంతని** bill చేస్తుంది.

అంటే service company యొక్క revenue formula ఇది:

SERVICE COMPANY యొక్క REVENUE – ఇది నీ ఉద్యోగంలో ప్రతిదాన్నీ వివరిస్తుంది

రెండు business models – ఒకే పని, వేరే ఆర్థిక శాస్త్రం

PRODUCT COMPANY

Revenue = product అమ్మకాలు
ఒక engineer product ని మెరుగుపరిస్తే
లాభం నేరుగా పెరుగుతుంది
→ తక్కువ మంది, కఠిన filter, ఎక్కువ pay

SERVICE COMPANY

Revenue = billable hours × rate
ఒక engineer bill అవుతున్నంత కాలం
అతను లాభదాయకుడు
→ ఎక్కువ మంది, వేగవంతమైన hiring, తక్కువ pay

Revenue = (ఎంతమంది bill అవుతున్నారు) × (గంటకి rate) × (గంటలు)

లాభం = ఆ revenue – నీ జీతం. ఈ తేడానే **margin** అంటారు.

ఈ ఒక్క formula నుంచి వచ్చే పర్యవసానాలు

Hiring వేగంగా ఉంటుంది – కానీ pay band ఇరుకుగా ఉంటుంది (margin కాపాడాలి)
Communication technical depth అంత ముఖ్యం – నువ్వు client ముందు కూర్చుంటావు
Timesheet ఒక ఆచారం కాదు – అదే invoice
"Bench" అనేది ఒక ఖర్చు – అందుకే అది ఒత్తిడితో కూడుకున్నది

ఒక్క వాక్యంలో: product company నీ *ఆలోచన* ని కొంటుంది; service company నీ *సమయాన్ని* కొని, దానికి margin కలిపి client కి అమ్ముతుంది. రెండూ నిజాయితీ వ్యాపారాలే – కానీ ఏది ఎలా పనిచేస్తుందో తెలియకపోతే, నువ్వు తప్పు అంచనాలతో లోపలికి వెళ్తావు.

Real-life Scenario

> ఒక **restaurant** ని, ఒక **catering company** ని పోల్చుకో.
>
> Restaurant (product company) తన సొంత menu తయారుచేస్తుంది. Chef ఒక కొత్త dish కనిపెడితే, అది restaurant కి శాశ్వత ఆస్తి. కాబట్టి restaurant కి **సృజనాత్మక chef** కావాలి, మరియు అతనికి ఎక్కువ జీతం ఇస్తుంది.
>
> Catering company (service company) దగ్గర సొంత menu ఉండదు. Client "మాకు 300 మందికి ఈ menu కావాలి" అంటాడు. Catering company 12 మంది cooks ని పంపుతుంది. వాళ్ళకి కావలసింది – **client చెప్పింది, సమయానికి, నాణ్యతగా వండగలిగే** cooks. Cook ఒక కొత్త dish కనిపెడితే అది మంచిదే, కానీ ఈ నెల invoice మీద దానికి ఒక్క రూపాయి కూడా రాదు.
>
> ఇక్కడ చెప్పదలచుకున్నది "service company తక్కువ" అని **కాదు**. ఇది ఒక వేరే ఆట – మరియు ఆ ఆట నియమాలు తెలిస్తే నువ్వు అందులో చాలా బాగా గెలవగలవు.

దీని నుంచి వచ్చే ఆచరణాత్మక పరిణామాలు

విషయం	Product company	Service MNC
Interview lo బరువు	Algorithmic depth, system design	Fundamentals + **communication + project ownership**
Hiring వేగం	4–10 వారాలు, చాలా rounds	**1-3 వారాలు**, 2-4 rounds
Tech stack ఎంపిక	Team నిర్ణయిస్తుంది	**Client నిర్ణయిస్తాడు** (నువ్వు కాదు)
కొత్త tech	Product కి అవసరమైతే	Client ఒప్పుకుంటేనే
నీ విలువ కొలత	Impact (metrics మారాయా)	**Billability + client feedback**
Pay band	వెడల్పు, negotiate చేయదగినది	ఇరుకు, grade కి కట్టుబడినది
Job security	Product నడిస్తే ఉంటుంది	Account నడిస్తే ఉంటుంది
నేర్చుకునే అవకాశం	లోతు (ఒకే product)	వెడల్పు (**పలు domains**)

Gotchas (సాధారణ తప్పులు)

1. **"Service company lo ఏమీ నేర్చుకోలేం" అనుకోవడం.** తప్పు. నువ్వు 3 ఏళ్ళలో banking, insurance, retail – మూడు domains చూస్తావు. Product company lo 3 ఏళ్ళు ఒకే product. **వెడల్పు ఇక్కడ ఎక్కువ, లోతు అక్కడ ఎక్కువ.** ఏది కావాలో నువ్వు నిర్ణయించుకో.
2. **Product-company mindset తో interview కి వెళ్ళడం.** "నేను ఈ architecture ని propose చేస్తాను, ఈ library వాడతాను" అని చెప్తే – service company panel కి అది *risk* లా వినిపిస్తుంది, *talent* లా కాదు. వాళ్ళకి కావలసింది "client requirement ని అర్థం చేసుకుని, deadline lo deliver చేసేవాడు".
3. **Margin ని విస్మరించడం.** నీ CTC ₹12L అయితే, company నిన్ను client కి దాదాపు ₹20–28L కి bill చేస్తుంది. ఆ తేడానే company లాభం. **నీ negotiation కి పరిమితి ఎక్కడ ఉందో ఇది చెప్తుంది** – company కి కావలసింది margin, నీ ప్రతిభ కాదు (ప్రతిభ rate ని పెంచినప్పుడు తప్ప).

Key Points

- Service company = **మనుషుల సమయాన్ని margin తో అమ్మే వ్యాపారం.**

- అందుకే: వేగవంతమైన hiring, ఇరుకైన pay bands, communication కి అధిక ప్రాధాన్యత.
- Tech stack ని **client నిర్ణయిస్తాడు** – ఇది నీ career planning ని మారుస్తుంది (§48 చూడు).
- ఇది తక్కువ కాదు – ఇది **వేరే ఆట**. నియమాలు తెలిసినవాడు గెలుస్తాడు.

---

2. Billability, bench, allocation, utilization – నీ career ని నడిపే నాలుగు మాటలు

వివరణ

ఈ నాలుగు పదాలు నీ offer letter lo ఉండవు, కానీ నీ రోజువారీ జీవితాన్ని, నీ appraisal ని, నీ promotion ని ఇవే నిర్ణయిస్తాయి. చాలామంది వీటిని మొదటి ఏడాది తర్వాత – నష్టపోయాక – నేర్చుకుంటారు.

Allocation = నువ్వు ఒక నిర్దిష్ట project కి కేటాయించబడ్డావు. Company system lo నీ పేరు ఆ project code కింద ఉంటుంది.

Billable = ఆ project కోసం నువ్వు పనిచేసే గంటలకి client కి invoice వెళ్తుంది. నువ్వు **company కి డబ్బు తెస్తున్నావు**.

Bench = నువ్వు ఏ project కీ కేటాయించబడలేదు. జీతం వస్తుంది, కానీ client కి invoice వెళ్ళదు. నువ్వు **company కి ఖర్చు**.

Utilization = ఒక కాలంలో నువ్వు billable గా ఉన్న శాతం. Managers దీన్ని కొలుస్తారు; వాళ్ళ appraisal కూడా దీని మీద ఆధారపడుతుంది.

నీ CAREER ని నడిపే నాలుగు స్థితులు

ఒక engineer యొక్క నాలుగు స్థితులు – వీటి మధ్య నువ్వు తిరుగుతూ ఉంటావు

Billable	▶	Shadow / KT	▶	Bench (కొత్తది)	▶	Bench (పాతది)
Project lo ఉన్నావు Invoice వెళ్తోంది సురక్షితం · ఇదే లక్ష్యం		Project lo ఉన్నావు కానీ ఇంకా bill కావట్లేదు సాధారణంగా 2-4 వారాలు		Project లేదు కానీ interviews వస్తున్నాయి ఇది సాధారణం · భయపడొద్దు		8+ వారాలు project లేదు Interviews తగ్గుతున్నాయి **ఇక్కడే ప్రమాదం**

Utilization = (billable గంటలు ÷ మొత్తం అందుబాటు గంటలు) × 100

Manager లక్ష్యం సాధారణంగా 80-90%. నువ్వు bench lo ఉంటే అతని సంఖ్య పడిపోతుంది –
అందుకే నిన్ను ఏదో ఒక project lo పెట్టడానికి అతను ఆతురతగా ఉంటాడు. ఇది నీకు అనుకూలం, ప్రతికూలం రెండూ.

Bench lo నువ్వు చేయాల్సినది	Bench lo నువ్వు చేయకూడనిది
రోజూ RMG / manager కి follow-up Internal interviews కి "అవును" అను Certification పూర్తిచేయి – ఇది allocation ని వేగవంతం చేస్తుంది	మౌనంగా ఎదురుచూడటం "మంచి project వచ్చేదాకా ఆగుతాను" అనుకోవడం మొదటి rejection తర్వాత ఆగిపోవడం

Bench గురించి నిజం: కొద్ది కాలం bench సాధారణం – project ముగిసినప్పుడు, rollout అయినప్పుడు అందరికీ వస్తుంది. **పొడవైన** bench ప్రమాదకరం: కొన్ని కంపెనీల్లో దీర్ఘకాలిక bench "performance improvement" సంభాషణకి దారితీస్తుంది. సంఖ్య కంపెనీని బట్టి మారుతుంది – కాబట్టి **నీ కంపెనీ policy ని join అయిన మొదటి వారంలోనే** అడిగి తెలుసుకో, bench lo పడ్డాక కాదు.

Real-life Scenario

> నువ్వు ఒక **taxi driver** అనుకో, కానీ taxi company నీకు నెలజీతం ఇస్తుంది.
>
> **Billable** = నీ cab lo passenger ఉన్నాడు, meter నడుస్తోంది. Company కి డబ్బు వస్తోంది. **Shadow/KT** = నువ్వు కొత్త city lo route నేర్చుకుంటున్నావు. Meter నడవట్లేదు, కానీ ఇది అవసరమైన పెట్టుబడి. **Bench** = నువ్వు stand lo నిలబడి ఉన్నావు. జీతం వస్తోంది, meter నడవట్లేదు.
>
> ఇప్పుడు manager దృష్టి నుంచి చూడు: అతని bonus "నా drivers ఎంత శాతం సమయం meter నడిపారు?" అనే దాని మీద ఆధారపడి ఉంది. కాబట్టి నువ్వు stand lo నిలబడితే **అతనికి కూడా నష్టం**.
>
> ఇది తెలిస్తే ఒక వ్యూహం స్పష్టమవుతుంది: **bench lo ఉన్నప్పుడు నీ manager నీ శత్రువు కాదు, నీ మిత్రుడు.** అతనికి కూడా నువ్వు allocate కావాలి. కాబట్టి "ఏదైనా opening ఉందా" అని అతన్ని అడగడం విసిగించడం కాదు — అది అతనికి సహాయం.

Gotchas (సాధారణ తప్పులు)

1. **"మంచి project వచ్చేదాకా ఆగుతాను" అనుకోవడం.** Bench lo నీ bargaining power **ప్రతి వారం తగ్గుతుంది**. మొదటి 2-3 వారాల్లో నీకు ఎంపిక ఉంటుంది; 10వ వారంలో నీకు మిగిలేది ఎవరూ తీసుకోని project.
2. **Internal interview ని తేలికగా తీసుకోవడం.** Account manager తీసుకునే internal interview కూడా ఒక నిజమైన interview. ఇక్కడ reject అయితే అది record అవుతుంది.
3. **Utilization ని విస్మరించడం.** Appraisal సమయంలో "నేను bench lo ఉన్నది నా తప్పు కాదు" అన్నది నిజం కావొచ్చు — కానీ సంఖ్య మాత్రం నీ పేరు మీదే ఉంటుంది. **Bench కాలంలో చేసిన certification, internal POC, hackathon — ఇవి ఆ ఖాళీని పూరిస్తాయి.** Document చెయ్యి.
4. **Timesheet ని ఆలస్యంగా నింపడం.** Timesheet నుంచే invoice తయారవుతుంది. ఆలస్యం = invoice ఆలస్యం = manager కి నేరుగా ఇబ్బంది. ఇది చిన్న విషయంలా కనిపిస్తుంది, కానీ నీ మీద అభిప్రాయాన్ని ఆశ్చర్యకరంగా ఎక్కువ ప్రభావితం చేస్తుంది.

Key Points

- **Billable = సురక్షితం.** Bench = ఖర్చు. ఇదే మొత్తం వ్యవస్థ యొక్క తర్కం.
- Bench lo **చొరవ** తీసుకున్నవాడు 3 వారాల్లో బయటపడతాడు; ఎదురుచూసినవాడు 3 నెలలు ఉంటాడు.
- నీ manager యొక్క ప్రోత్సాహకం నీదానితో **కలిసే ఉంటుంది** — అతన్ని వాడుకో.
- **Timesheet = invoice.** దీన్ని ఎప్పుడూ తేలికగా తీసుకోకు.

---

3. Account, project, pod — ఎవరు ఎవరు

వివరణ

Service company యొక్క నిర్మాణం product company కంటే వేరుగా ఉంటుంది, మరియు ఈ తేడా తెలియకపోతే నువ్వు **తప్పు మనిషిని తప్పు ప్రశ్న అడుగుతావు**.

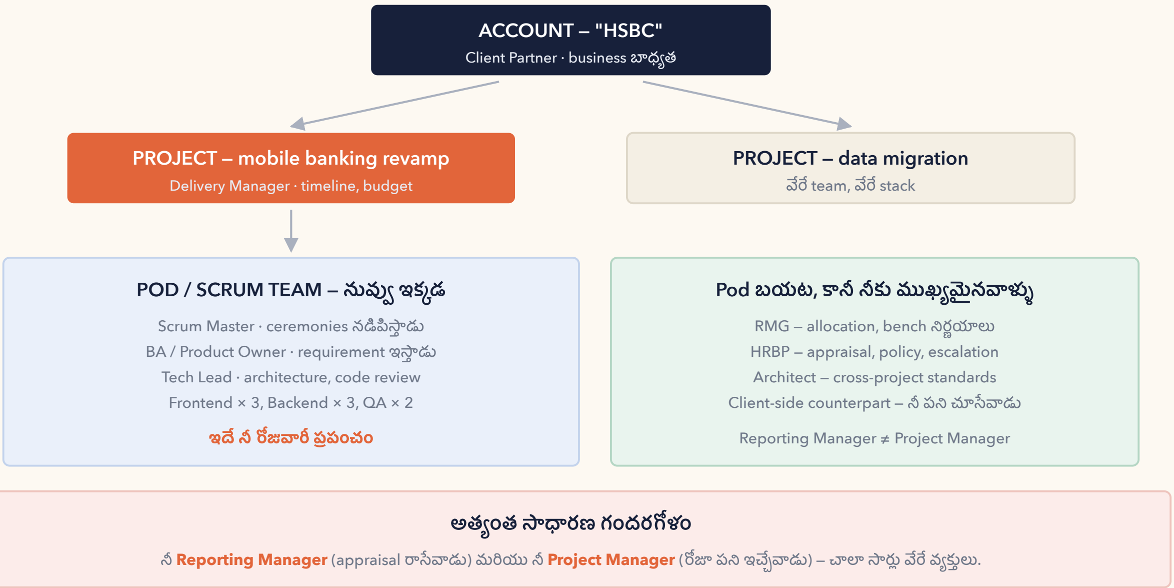
- **Account** = ఒక client. ఉదాహరణకి "HSBC account", "Walmart account". ఒక account lo వందల మంది ఉండొచ్చు, పలు projects ఉండొచ్చు. దీన్ని ఒక **Client Partner / Account Manager** నడిపిస్తాడు

– ఇతను business మనిషి, technical కాదు.

- **Project / Engagement** = ఆ account lo ఒక నిర్దిష్ట పని. "HSBC – mobile banking revamp". దీనికి ఒక **Delivery Manager / Project Manager** ఉంటాడు.
- **Pod / Scrum team / Squad** = 6–10 మంది, నువ్వు రోజూ పనిచేసేది వీళ్ళతోనే.

నీ చుట్టూ ఉన్న మనుషులు · ఎవరు దేనికి బాధ్యులు

ACCOUNT → PROJECT → POD – ఒక పెద్ద MNC lo నిర్మాణం



ఈ చివరి point ని తక్కువ అంచనా వెయ్యొద్దు. నీ appraisal రాసేవాడు నీ రోజువారీ పని **చూడకపోవచ్చు** – అతను project manager నుంచి feedback తీసుకుంటాడు. అంటే నీ పని **ఇద్దరికీ** కనిపించాలి. §40 lo దీని గురించి వివరంగా.

Real-life Scenario

> ఒక పెద్ద **construction company** ని ఊహించుకో.
>
> **Account** = "మేము Prestige Group కి కట్టిస్తున్నాము" – ఒక client సంబంధం. **Project** = "Prestige Lakeside – Tower B" – ఒక నిర్దిష్ట భవనం. **Pod** = నువ్వు రోజూ పనిచేసే 8 మంది బృందం.
>
> ఇప్పుడు కీలకమైన విషయం: నీ **site engineer** (project manager) రోజూ నీ పని చూస్తాడు. కానీ నీ **జీతం పెంపు** రాసేది company lo ఉన్న వేరే **department head** (reporting manager) – అతను site కి నెలకోసారి వస్తాడు.
>
> కాబట్టి "నేను బాగా పనిచేశాను, site engineer కి తెలుసు" అన్నది సరిపోదు. **అది department head కి చేరాలి.** అది దానంతట అదే జరగదు – నువ్వు చేయాలి.

ఎవరిని ఏమి అడగాలి

నీ ప్రశ్న	ఎవరిని అడగాలి
"ఈ ticket lo requirement ఏమిటి?"	BA / Product Owner
"ఈ approach సరైనదేనా?"	Tech Lead
"Sprint scope మార్చొచ్చా?"	Scrum Master → PO

నీ ప్రశ్న	ఎవరిని అడగాలి
"నాకు ఈ project నచ్చట్లేదు, మార్చొచ్చా?"	Reporting Manager (**PM కాదు**)
"నేను bench lo ఉన్నాను, opening ఏమైనా ఉందా?"	RMG + Reporting Manager
"నా promotion ఎప్పుడు?"	Reporting Manager
"Client ఈ pending item గురించి అడుగుతున్నాడు"	Project Manager
"Policy / leave / escalation"	HRBP

Gotchas (సాధారణ తప్పులు)

1. **అన్ని ప్రశ్నలూ Tech Lead ని అడగడం.** Tech Lead కి architecture ప్రశ్నలు. Requirement ప్రశ్నలు BA వి. తప్పు మనిషిని అడిగితే నెమ్మదిగా "ఇతనికి clarity లేదు" అనే అభిప్రాయం ఏర్పడుతుంది.
2. **Reporting Manager తో సంబంధం లేకుండా ఉండటం.** అతనే నీ rating రాస్తాడు. నెలకి ఒక్కసారైనా 15 నిమిషాల 1:1 – ఇది ఐచ్ఛికం కాదు.
3. **Client-side counterpart ని విస్మరించడం.** చాలా projects lo client వైపు ఒక tech lead ఉంటాడు. అతని మంచి అభిప్రాయం నీకు **onsite అవకాశం** తెస్తుంది; చెడ్డ అభిప్రాయం నిన్ను project నుంచి బయటకి పంపిస్తుంది.

Key Points

- Account (client) → Project (పని) → Pod (నీ team). మూడు వేరే స్థాయిలు.
- **Reporting Manager ≠ Project Manager.** ఇది చాలా MNCs lo నిజం.
- నీ పని **ఇద్దరికీ** కనిపించాలి – లేకపోతే appraisal lo ఖాళీ ఉంటుంది.
- RMG = allocation యొక్క తలుపు. బెంచ్ lo ఉన్నప్పుడు ఈ పేరు నీకు తెలియాలి.

---

4. Frontend engineer రోజూ ఏం చేస్తాడు – JD చెప్పేది vs నిజం

వివరణ

Job description lo "React, Redux, TypeScript, REST APIs, Agile" అని ఉంటుంది. అది తప్పు కాదు, కానీ అది **పని యొక్క 40% మాత్రమే**. మిగతా 60% ఏమిటో ముందే తెలిస్తే – నువ్వు interview lo మెరుగ్గా మాట్లాడతావు, మరియు join అయ్యాక ఆశ్చర్యపోవు.

ఒక FRONTEND ENGINEER యొక్క నిజమైన సమయ విభజన

JD చెప్పేది vs వారంలో సమయం నిజంగా ఎక్కడికి పోతుంది

కొత్త feature code రాయడం	~40% – JD lo ఉన్నది ఇదే
Bug fix	~18% – defect triage, UAT fixes, regression
Meetings	~15% – standup, grooming, demo, client call
Review	~12% – PR review ఇవ్వడం, తీసుకోవడం
Docs	~10% – design doc, KT notes, RCA, timesheet
	~5% – environment / build / access సమస్యలు

ఇక్కడ దాగి ఉన్న పాఠం

Code రాయడం మాత్రమే బాగా చేసేవాడు "మంచి developer". Code + communication + documentation బాగా చేసేవాడు **promote అవుతాడు**. Service company lo ఈ తేడా చాలా పెద్దది.

ఈ శాతాలు ఒక సాధారణ development project కి. **Support project** అయితే bug fix 50%+ కి పెరుగుతుంది, కొత్త feature 10% కి పడిపోతుంది – అందుకే §47 lo ఆ తేడా గురించి ప్రత్యేకంగా చెప్పాను.

ఒక నిజమైన ticket జీవితచక్రం

1. **BA** ఒక user story రాస్తాడు: *"As a customer, I want to filter transactions by date range."*
2. **Grooming** meeting lo team దాన్ని చర్చిస్తుంది. నువ్వు అడుగుతావు: "Date range కి గరిష్ఠ పరిమితి ఉందా? API pagination support చేస్తుందా? Timezone client దా server దా?" – **ఈ ప్రశ్నలు అడగడమే నిన్ను వేరు చేస్తుంది.**
3. Story కి **points** ఇస్తారు (3, 5, 8...). నువ్వు sprint lo దాన్ని తీసుకుంటావు.
4. నువ్వు **branch** తీసి code రాస్తావు. Component, state, API call, loading/error states, unit tests.
5. **PR** పెడతావు. Tech Lead review చేస్తాడు. 2-3 rounds comments.
6. **QA** dev environment lo test చేస్తుంది. 2 defects raise చేస్తుంది. నువ్వు fix చేస్తావు.
7. **Demo** lo client కి చూపిస్తారు. Client అంటాడు: "బాగుంది కానీ default గా చివరి 30 రోజులు చూపించాలి."
8. అది **కొత్త ticket** అవుతుంది (లేదా scope creep గా వాదన జరుగుతుంది).
9. **UAT** lo client team test చేస్తుంది. ఇంకో defect.
10. **Release** – production కి వెళ్తుంది. Release notes, deployment checklist.

> **ఇక్కడ గమనించాల్సినది:** step 4 (code రాయడం) మొత్తం ప్రక్రియలో **ఒక్క అడుగు**. మిగతా 9 అడుగులూ – ప్రశ్నలు అడగడం, వివరించడం, సమన్వయం చేయడం. అందుకే service company interviews lo communication కి అంత బరువు ఉంటుంది.

JD lo కనిపించని, కానీ రోజూ చేసే పనులు

పని	ఎందుకు ముఖ్యం
Requirement lo ఖాళీలు కనిపెట్టడం	BA ప్రతి edge case రాయలేడు. నువ్వు అడగకపోతే, అది defect అవుతుంది

పని	ఎందుకు ముఖ్యం
Status ని స్పష్టంగా చెప్పడం	"దాదాపు అయిపోయింది" అనేది standup lo విషం. "API integration అయ్యింది, error states మిగిలాయి, రేపు మధ్యాహ్నానికి" – ఇది బంగారం
KT (Knowledge Transfer)	ఎవరైనా project వదిలితే, మిగిలినవాళ్ళు నేర్చుకోవాలి. KT documents రాయడం, ఇవ్వడం
Estimation	"ఇది ఎన్ని రోజులు?" – ఇది ఒక నైపుణ్యం. తక్కువ చెప్తే నువ్వు ఇబ్బంది పడతావు; ఎక్కువ చెప్తే నమ్మకం పోతుంది
Defect triage	Bug నీదా, backend దా, data దా – దీన్ని త్వరగా, రాజకీయం లేకుండా తేల్చడం
Environment సమస్యలు	VPN, proxy, certificate, access – MNC lo ఇవి నిజమైన సమయం తినేస్తాయి

Gotchas (సాధారణ తప్పులు)

1. **మౌనంగా పనిచేయడం.** Service company lo *కనిపించని పని* = *జరగని పని*. నువ్వు 3 రోజులు ఒక క్లిష్టమైన bug మీద పనిచేసి, ఎవరికీ చెప్పకపోతే – record lo నువ్వు 3 రోజులు ఏమీ చేయలేదు.
2. **"నాకు అర్థం కాలేదు" అని అడగకపోవడం.** మొదటి వారంలో అడిగితే అది నేర్చుకోవడం. మూడో వారంలో అడిగితే అది గందరగోళం. **ముందే అడుగు.**
3. **Estimate ని ఒత్తిడిలో తగ్గించడం.** PM "2 రోజుల్లో వస్తుందా?" అంటే, నిజం 4 రోజులైతే 4 అనే చెప్పు. ఒకసారి తప్పు estimate ఇస్తే అది నీ మీదే పడుతుంది.
4. **Frontend-only ఆలోచించడం.** "API slow, అది నా పని కాదు" – ఇది technically నిజం కావొచ్చు, కానీ client కి కనిపించేది **నీ screen**. Backend team తో కలిసి పరిష్కరించేవాడే విలువైనవాడు.

Key Points

- Code రాయడం పని lo **~40%**. మిగతాది సమన్వయం, స్పష్టత, documentation.
- ఒక ticket lo **10 అడుగులు** ఉంటాయి; coding ఒక్కటే.
- **కనిపించని పని = జరగని పని.** Standup, PR description, demo – ఇవే నీ కనిపించే ముఖం.
- Requirement lo ఖాళీలు కనిపెట్టడం – ఇదే 2 సంవత్సరాల engineer ని 5 సంవత్సరాల engineer నుంచి వేరు చేస్తుంది.

Part 1 – Levels, Designations, Pay

5. ఏడు కంపెనీల level ladder – ఒక పోలిక పట్టిక

వివరణ

ప్రతి service MNC కి తన సొంత **grade code** ఉంటుంది – JL3A, C1, B2, CL10, A4. ఇవి బయటివాళ్ళకి అర్థం కావు, మరియు internet lo ఉన్న సగం సమాచారం **తప్పు**. ఈ codes ముఖ్యం ఎందుకంటే – నీ promotion cycle, onsite eligibility, ఏ role కి apply చేయగలవు – అన్నీ వీటి మీదే ఆధారపడి ఉంటాయి.

ఈ పట్టికని ఎలా చదవాలి. ఈ డేటా ఉద్యోగులు స్వయంగా report చేసిన మూలాల నుంచి (Levels.fyi, Glassdoor, Fishbowl, Grapevine, Quora). ఏ కంపెనీ కూడా దీన్ని అధికారికంగా ప్రచురించదు, మరియు bands business unit ని బట్టి, నియామక సంవత్సరాన్ని బట్టి మారతాయి. కాబట్టి ఇది **దిక్సూచి, పటం కాదు**. నీ నిర్దిష్ట offer lo grade ని recruiter ని అడిగి నిర్ధారించుకో.

SERVICE MNC LEVEL LADDERS · 0-10 సంవత్సరాల పరిధిలో

ఒకే అనుభవం, ఏడు వేరే పేర్లు – సుమారు సమాంతర స్థాయిలు

	0-2 సంవత్సరాలు	2-5 సంవత్సరాలు	5-8 సంవత్సరాలు	8-12 సంవత్సరాలు
Infosys	JL3 · Systems Engineer	**JL3A · Senior SE**	JL4 · Technology Analyst	JL5 · Tech Lead
TCS	C1Y · ASE	**C1 · Systems Engineer**	C2 · IT Analyst	C3A · Assistant Consultant
Wipro	A2 / B1 · SE	**B2 · Senior SE**	B3 · Technical Lead	C1 · Project Manager
Accenture	CL12 / CL11 · Analyst	**CL10 · Senior Analyst**	CL9 · Team Lead	CL8 · Associate Manager
Capgemini	A3 / A4 · SE	**A5 · Senior SE**	B1 · Associate Consultant	B2 · Consultant
Cognizant	Programmer Analyst	**Associate**	Senior Associate	Manager · Projects
LTIMindtree	Software Engineer	**Senior SE**	C3 · Technical Lead	Module Lead

Accenture గురించి ఒక ఉచ్చు

CL సంఖ్య **వెనక్కి** నడుస్తుంది – CL13 అత్యంత junior, CL1 అత్యంత senior. CL9, CL11 కంటే పెద్దది.

Infosys గురించి ఒక ఉచ్చు

చాలా blogs "JL5 = 5 సంవత్సరాలు" అని రాస్తాయి. అది తప్పు – JL5/JL6 అనేవి **10-15+ సంవత్సరాల** Tech Lead / Architect స్థాయిలు.

HCLTech (E0/E1/E2 తరహా grades) మరియు merger తర్వాత LTIMindtree యొక్క ఏకీకృత band mapping గురించి **నమ్మదగిన బహిరంగ డేటా దొరకలేదు** – అందుకే ఆ రెండిటిని ఇక్కడ designation పేర్లతో మాత్రమే చూపించాను, grade codes తో కాదు. ఆ కంపెనీల offer వస్తే grade code ని recruiter ని నేరుగా అడుగు.

పూర్తి పట్టిక – designation పేర్లతో

సంవత్సరాలు	Infosys	TCS	Wipro	Accenture	Capgemini
0-2	JL3 – Systems Engineer	C1Y – ASE	B1 – Software Engineer	CL12/11 – Analyst	A3/A4 – (Assoc.) SE
2-5	**JL3A – Senior Systems Engineer**	**C1 – Systems Engineer**	**B2 – Senior SE**	**CL10 – Senior Analyst**	**A5 – Senior SE**
5-8	JL4 – Technology Analyst	C2 – IT Analyst	B3 – Technical Lead	CL9 – Team Lead / Specialist	B1 – Associate Consultant
8-12	JL5 – Technology Lead	C3A – Assistant Consultant	C1 – Project Manager	CL8 – Associate Manager	B2 – Consultant
12+	JL6 – Senior Tech Lead / Architect	C3B – Associate Consultant	C2 – Senior PM	CL7 – Manager	C1 – Senior Consultant

Gotchas (సాధారణ తప్పులు)

1. **Designation ని కంపెనీల మధ్య పోల్చడం.** Wipro "Senior Software Engineer" (B2, ~3 సం.) మరియు Capgemini "Senior Software Engineer" (A5) దాదాపు సమానం – కానీ Mindtree "Technical Lead" (C3) ~4.5 సం. కి కూడా రావొచ్చు. **Designation కాదు, grade code మరియు pay band** చూడు.
2. **Accenture CL సంఖ్యని ముందుకి చదవడం.** CL9 > CL10 > CL11. ఒక recruiter "CL10 ఇస్తున్నాం" అంటే అది CL11 కంటే **మెరుగు**.
3. **Internet blogs ని నమ్మడం.** ముఖ్యంగా Infosys JL levels గురించి – చాలా blogs JL5 ని 5 సంవత్సరాలకి mapping చేస్తాయి. అది తప్పు, మరియు ఆ తప్పు అంచనాతో negotiate చేస్తే నువ్వు నవ్వులపాలవుతావు.
4. **Grade ని offer letter lo చూడకపోవడం.** Offer letter lo grade code స్పష్టంగా ఉండాలి. లేకపోతే **రాతపూర్వకంగా అడుగు**. Verbal హామీలు appraisal సమయంలో మాయమవుతాయి.

Key Points

- ప్రతి కంపెనీకి సొంత code; **designation కాదు, grade + pay చూడు**.
- 2-5 సంవత్సరాల band: Infosys **JL3A**, TCS **C1**, Wipro **B2**, Accenture **CL10**, Capgemini **A5**.
- Accenture CL సంఖ్య **వెనక్కి** నడుస్తుంది.
- Infosys **JL5 = 10+ సంవత్సరాలు**, 5 కాదు.

---

6. నీ experience కి ఏ level వస్తుంది

వివరణ

పైన ఉన్నది సాధారణ mapping. కానీ నీ నిర్దిష్ట offer ని నిర్ణయించేది అనుభవం ఒక్కటే కాదు – **నాలుగు విషయాలు** కలిసి నిర్ణయిస్తాయి.

OFFER LO నీ GRADE ని నిర్ణయించే నాలుగు అంశాలు

ఇవి కలిసి నీ grade ని, తద్వారా నీ CTC పరిమితిని నిర్ణయిస్తాయి

1 · అనుభవం	2 · ప్రస్తుత CTC	3 · Requisition band	4 · Panel feedback
సంవత్సరాలు – కానీ ఇది కేవలం ప్రవేశ ద్వారం	Lateral offer దాదాపు ఎప్పుడూ దీని మీద 20-40% hike	ఆ req ఏ grade కి తెరిచారో దాని పైన ఇవ్వలేరు	"ఇతను ఒక level పైన" అని panel రాస్తేనే
బరువు: ~30%	బరువు: ~35%	బరువు: ~25%	బరువు: ~10%

దీని నుంచి వచ్చే అతి ముఖ్యమైన వ్యూహం

నీ grade కి **అతిపెద్ద ఒకే కారణం నీ ప్రస్తుత CTC** – నీ ప్రతిభ కాదు.
అంటే: తక్కువ CTC నుంచి వచ్చేవాడికి, అదే skill ఉన్నా, తక్కువ offer వస్తుంది.
దీన్ని విరగ్గొట్టే ఏకైక మార్గం – వేరే offer చూపించడం (§38).

Requisition band అనే గోడ

ఒక req "JL3A" కి తెరిచి ఉంటే, నువ్వు ఎంత బాగా చేసినా JL4 ఇవ్వలేరు – అది వేరే బడ్జెట్ రేఖ.
కాబట్టి recruiter screen lo **"ఈ position ఏ grade కి తెరిచారు?"** అని అడగడం –
ఇది దురుసుతనం కాదు, ఇది నీ సమయాన్ని కాపాడుకోవడం.

అనుభవం → grade → CTC (2026, సూచన మాత్రమే)

అనుభవం	సాధారణ grade	Frontend/React CTC పరిధి	గమనిక
0-2 సం.	Entry (JL3, C1Y, B1, CL11)	₹3.5-6.5 L	Fresher pipeline; campus offers తక్కువ చివర
2-4 సం.	**JL3A, C1, B2, CL10, A5**	**₹8-14 L**	ఇక్కడే lateral hiring అత్యధికం
4-7 సం.	JL4, C2, B3, CL9, B1	₹14-22 L	Lead బాధ్యతలు మొదలవుతాయి
7-10 సం.	JL5, C3A, C1, CL8, B2	₹20-32 L	Team/delivery బాధ్యత

ఈ సంఖ్యల మూలం. Naukri listing విశ్లేషణ 3-5 సంవత్సరాల React roles కి median **₹8-14 LPA** అని చూపిస్తుంది (Angular కి ₹7-12 LPA). Levels.fyi Capgemini India Software Engineer median ₹7.2L (A3 ₹4.33L నుంచి C1 ₹17.1L వరకు), Cognizant frontend median total comp ₹7.89L. Accenture India కి Glassdoor forum సూచించిన fixed-pay bands: CL11 ₹6-9L, CL10 ₹9-15L, CL9 ₹15-22L. ఇవన్నీ **స్వయం-report చేసిన** సంఖ్యలు – location, negotiation, demand ని బట్టి ±30% కదులుతాయి.

Real-life Scenario

> ఇద్దరు candidates – **రవి** మరియు **కిరణ్** – ఇద్దరూ 3 సంవత్సరాల React అనుభవం, దాదాపు ఒకే skill.
>
> **రవి**: ప్రస్తుత CTC ₹6L (ఒక చిన్న startup lo). Offer: ₹8.4L (40% hike). Grade JL3A. **కిరణ్**: ప్రస్తుత CTC ₹11L (ఒక product company lo). Offer: ₹14.3L (30% hike). Grade JL3A – **అదే grade**.
>
> ఒకే grade, ఒకే పని, ₹6L తేడా. ఇది అన్యాయమా? అవును. ఇది వాస్తవమా? కూడా అవును.
>
> **రవి కి ఉన్న ఏకైక మార్గం**: ఇంకో offer సంపాదించి దాన్ని చూపించడం. "నా ప్రస్తుత CTC ₹6L, కానీ నా దగ్గర ₹12L offer ఉంది" – ఇది సంభాషణని పూర్తిగా మారుస్తుంది, ఎందుకంటే ఇప్పుడు **మార్కెట్ rate** ఒక సాక్ష్యంగా మారింది, నీ గత జీతం కాదు.

Gotchas (సాధారణ తప్పులు)

1. **ప్రస్తుత CTC ని మొదటి call lo చెప్పేయడం.** అది చెప్పిన క్షణం, నీ offer యొక్క పైకప్పు నిర్ణయమైపోయింది. వీలైనంత ఆలస్యం చెయ్యి; "నేను మార్కెట్ rate చూస్తున్నాను, role మరియు grade తెలిస్తే expectation చెప్పగలను" అనేది ఒక న్యాయమైన సమాధానం.
2. **Grade ని అడగకపోవడం.** "ఈ req ఏ level కి?" – ఇది recruiter screen lo అడగాల్సిన మొదటి మూడు ప్రశ్నల్లో ఒకటి.
3. **Hike శాతం మీద మాత్రమే దృష్టి పెట్టడం.** 40% hike బాగుంది – కానీ base తక్కువైతే 40% కూడా తక్కువే. **Absolute సంఖ్య** చూడు, శాతం కాదు.
4. **"అనుభవం పెరిగితే grade దానంతట అదే వస్తుంది" అనుకోవడం.** Internal promotion **నెమ్మది** (§7). Wipro lo B1→B2 కి ~3.5 సంవత్సరాలు పట్టిన ఉదాహరణలు ఉన్నాయి.

Key Points

- Grade ని నిర్ణయించేది: అనుభవం (~30%) + **ప్రస్తుత CTC (~35%)** + req band (~25%) + panel (~10%).
- 2-4 సంవత్సరాల frontend/React: **₹8-14 L** అనేది వాస్తవిక పరిధి.
- **Requisition band ఒక గట్టి గోడ** – దాని పైన ఇవ్వలేరు, ఎంత బాగా చేసినా.
- ప్రస్తుత CTC ఉన్నప్పుడు వ్యతిరేకంగా ఏకైక ఆయుధం – **ఇంకో offer**.

---

7. Lateral hire vs internal promotion – laterals grades ఎందుకు దూకుతారు

వివరణ

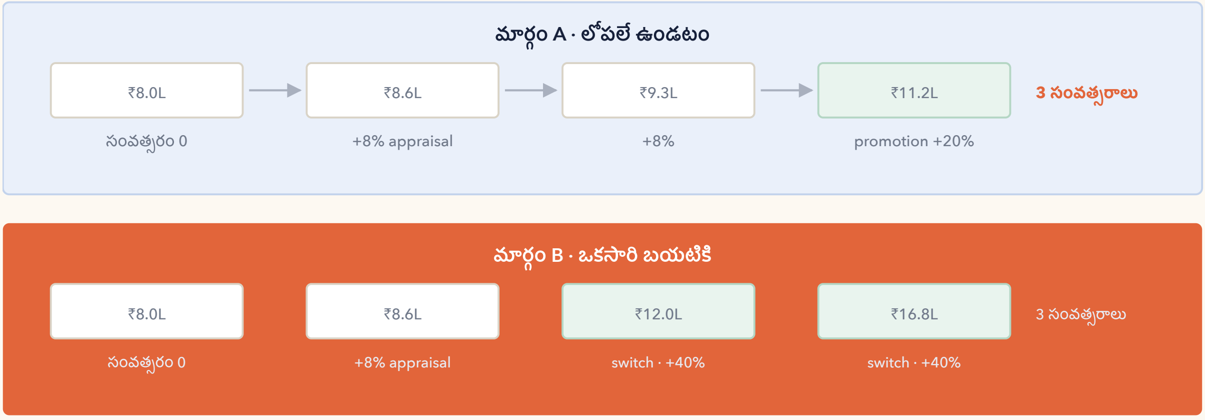
Service MNC lo ఒక చేదు నిజం ఉంది, మరియు దాన్ని ముందే తెలుసుకోవడం మంచిది:

> **లోపల ఉండి పెరగడం కంటే, బయటికి వెళ్ళి తిరిగి రావడం వేగంగా పని చేస్తుంది.**

కారణం structural, వ్యక్తిగతం కాదు. Internal promotion ఒక **budget cycle** లో ఇరుక్కుని ఉంటుంది – ప్రతి సంవత్సరం ఒక నిర్దిష్ట శాతం మందిని మాత్రమే promote చేయగలరు, మరియు అది bell curve కి కట్టుబడి ఉంటుంది. Lateral hiring ఒక **demand cycle** నుంచి వస్తుంది – ఒక account కి 20 React developers వెంటనే కావాలంటే, మార్కెట్ rate ఇవ్వక తప్పదు.

రెండు మార్గాలు · ఒకే గమ్యం, వేరే వేగం

₹8L నుంచి ₹16L కి – రెండు మార్గాలు



కానీ మార్గం B కి ఒక ధర ఉంది – ప్రతి switch lo నువ్వు domain knowledge, సంబంధాలు, notice period, మరియు కొంత స్థిరత్వాన్ని కోల్పోతావు.

ఈ సంఖ్యలు ఉదాహరణ కోసం – కానీ నమూనా నిజమైనది. **Appraisal hikes సాధారణంగా 5-12%**, **switch hikes 20-40%**. ఈ అంతరమే ప్రతి సంవత్సరం లక్షల మందిని కంపెనీలు మార్చేలా చేస్తుంది.

అయితే ప్రతి సంవత్సరం మారాలా? – లేదు

ఇది ఒక ఉచ్చు. చాలా మంది ఈ గణితాన్ని చూసి "ప్రతి 18 నెలలకి మారుతాను" అనుకుంటారు. దానితో వచ్చే సమస్యలు:

సమస్య	ఏమవుతుంది
Resume పరిశీలన	3 కంపెనీలు 4 సంవత్సరాల్లో – recruiters దీన్ని "job hopper" అని filter చేస్తారు, ముఖ్యంగా service MNCs

సమస్య	ఏమవుతుంది
లోతు లేకపోవడం	ప్రతి చోటా 12 నెలలు = ఎక్కడా ఒక పూర్తి product cycle చూడలేదు. Interview lo ఇది కనిపిస్తుంది
Notice period	Service MNCs lo 60-90 రోజులు. ప్రతి మార్పుకి 3 నెలల ఇబ్బంది
Promotion అర్హత	చాలా కంపెనీల్లో promotion కి **కనీసం 12-18 నెలల** tenure అవసరం. తొందరగా మారితే నువ్వు ఎప్పుడూ ఆ గీత దాటవు

> **ఆచరణాత్మక నియమం:** మొదటి కంపెనీ lo **2-3 సంవత్సరాలు** (ఒక పూర్తి project cycle + ఒక appraisal + ఒక promotion ప్రయత్నం). తర్వాత switch. ఆ తర్వాత **3-4 సంవత్సరాలు**. ఇది hike ని, స్థిరత్వాన్ని రెండిటినీ కాపాడుతుంది.

లోపల ఉండే వ్యూహం – దాన్ని పనిచేయించాలంటే

లోపలే ఉండాలనుకుంటే, appraisal hike (~8%) మీద ఆధారపడొద్దు. ఈ మూడు మాత్రమే పెద్ద మార్పు తెస్తాయి:

1. **Internal promotion** – grade మారుతుంది, hike 15-25%.
2. **Onsite deputation** – allowance వల్ల effective pay చాలా పెరుగుతుంది (§42).
3. **Critical skill retention** – నువ్వు వదిలితే project ఆగిపోతుందంటే, counter-offer వస్తుంది. కానీ ఇది ఒక్కసారే పనిచేస్తుంది.

Gotchas (సాధారణ తప్పులు)

1. **Counter-offer ని అంగీకరించడం.** నువ్వు resign చేసినప్పుడు వచ్చే counter-offer – గణాంకపరంగా చాలా మంది 6-12 నెలల్లో మళ్ళీ వెళ్ళిపోతారు. ఎందుకంటే *ఎందుకు* వెళ్ళాలనుకున్నావో ఆ కారణం (project, manager, growth) మారలేదు – డబ్బు మాత్రమే మారింది.
2. **"Loyalty కి బహుమతి వస్తుంది" అనుకోవడం.** Service MNC lo loyalty కి బహుమతి **స్థిరత్వం**, డబ్బు కాదు. అది తక్కువ విలువైనది కాదు – కానీ దాన్ని డబ్బు అని పొరబడొద్దు.
3. **Boomerang ని మర్చిపోవడం.** అదే కంపెనీకి 2 సంవత్సరాల తర్వాత తిరిగి రావడం (boomerang hire) చాలా సార్లు internal promotion కంటే ఎక్కువ grade + pay ఇస్తుంది. ఇది వింతగా ఉంది, కానీ నిజం.

Key Points

- **Internal hike ~5-12%; switch hike ~20-40%.** ఈ అంతరం structural.
- కానీ **ప్రతి 18 నెలలకి మారడం** resume ని దెబ్బతీస్తుంది – 2-3 సంవత్సరాలు సరైన లయ.
- లోపల పెరగాలంటే: promotion, onsite, లేదా critical-skill retention. Appraisal ఒక్కటే సరిపోదు.
- Counter-offer సాధారణంగా ఒక తాత్కాలిక పరిష్కారం.

---

8. Offer lo నీ level ని నిజంగా ఏది నిర్ణయిస్తుంది

వివరణ

§6 lo నాలుగు అంశాలు చూశాం. ఇక్కడ వాటిని **నువ్వు ఏమి నియంత్రించగలవు, ఏమి నియంత్రించలేవు** అని విడగొడదాం – ఎందుకంటే శక్తిని నియంత్రించగలిగే వాటి మీద పెట్టాలి.

అంశం	నీ నియంత్రణ	ఏం చేయగలవు
అనుభవం (సంవత్సరాలు)	సున్నా	ఏమీ లేదు – కానీ ఏ రకమైన అనుభవమో దాన్ని presentation తో మార్చగలవు
ప్రస్తుత CTC	తక్కువ	చెప్పే **సమయాన్ని** ఆలస్యం చేయగలవు; variable/bonus కలిపి మొత్తం చెప్పగలవు
Requisition band	సున్నా	కానీ **ఏ req కి apply చేయాలో ఎంచుకోగలవు** – ఇదే అతిపెద్ద lever
Panel feedback	ఎక్కువ	ఇక్కడే నీ ప్రయత్నం విలువ చేస్తుంది
పోటీ offer	ఎక్కువ	ఒకేసారి 3-4 కంపెనీలకి apply చేయడం

> **సారాంశం:** నీ grade ని పెంచే రెండు నిజమైన levers – **సరైన requisition ఎంచుకోవడం** మరియు **పోటీ offer**. మిగతావన్నీ అంచుల్లో కదులుతాయి.

Panel feedback lo "ఒక level పైన" అని ఎప్పుడు రాస్తారు

Interview panel సాధారణంగా ఒక form నింపుతుంది: *Recommended level: below / at / above requisition*. "Above" రావాలంటే ఈ మూడు కనిపించాలి:

1. **Ownership కథ** – నువ్వు ఒక feature ని end-to-end నడిపిన స్పష్టమైన ఉదాహరణ (requirement clarify నుంచి production వరకు).
2. **ఇతరులని ప్రభావితం చేయడం** – "నేను team కి code review checklist ప్రవేశపెట్టాను", "juniors కి onboarding doc రాశాను". ఇది lead-track సంకేతం.
3. **Trade-off ఆలోచన** – "మేము context API వాడొచ్చు, కానీ ఈ scale lo re-render ఖర్చు ఎక్కువ కాబట్టి..." – నిర్ణయం **ఎందుకు** తీసుకున్నావో చెప్పడం.

Panel ని "above" వైపు తిప్పే సమాధాన నమూనా (English – ఇలాగే చెప్పు):

"On the transactions module, the requirement was just 'add a date filter'. During grooming I asked whether the range needed a cap, because our API returns unpaginated results – a one-year range would have pulled about 40,000 rows into the browser. We agreed on a 90-day cap plus server-side pagination. I built the filter with React Query for caching, added the loading and empty states, and wrote the KT doc for it. The reason I raised it in grooming rather than in testing is that a schema change after QA would have cost us a sprint."

ఇందులో ఉన్నవి: requirement ని ప్రశ్నించడం, ఒక సంఖ్య (40,000), ఒక trade-off, ఒక పరిణామం (a sprint). **ఇదే "above" సంకేతం.**

Key Points

- నియంత్రించగలిగేవి: **ఏ req కి apply చేస్తావు** + **పోటీ offer** + **panel feedback**.
- Panel "above level" రాయాలంటే: ownership + ఇతరులపై ప్రభావం + trade-off ఆలోచన.
- కథ చెప్పేటప్పుడు ఎప్పుడూ **ఒక సంఖ్య + ఒక పరిణామం** ఉండాలి.

9. Level code ఎందుకు పట్టించుకోవాలి

వివరణ

"Grade ఏదైతే ఏమిటి, జీతం బాగుంటే చాలు" – ఇది join అయ్యే ముందు సహజమైన ఆలోచన, మరియు **ఏడాది తర్వాత ఖరీదైనదని తెలుస్తుంది**. Grade ఈ ఐదిటిని నియంత్రిస్తుంది:

Grade దేన్ని నియంత్రిస్తుంది	ఎలా
Promotion cycle	తర్వాతి grade కి కనీస tenure (సాధారణంగా 18-24 నెలలు) ఆ grade నుంచే లెక్కిస్తారు
Onsite eligibility	చాలా accounts lo onsite deputation కి కనీస grade ఉంటుంది
Role eligibility	"Tech Lead" role కి B3/JL4 కావాలి – నువ్వు ఎంత సమర్థుడైనా B2 అయితే ఆ role ఇవ్వలేరు
Variable pay శాతం	ఉన్నత grades lo variable భాగం పెద్దది (ఇది మంచిదా చెడ్డదా అనేది §10)
తర్వాతి కంపెనీ	తర్వాతి interview lo "మీ designation ఏమిటి?" – తక్కువ grade అంటే తక్కువ ప్రారంభ స్థానం

Real-life Scenario

> **అను** కి రెండు offers వచ్చాయి, ఒకే కంపెనీ నుంచి కాదు:
>
> - Offer 1: **₹13.5L**, grade **B2 (Senior Software Engineer)**
> - Offer 2: **₹14.2L**, grade **B1 (Software Engineer)** – "మా band lo ఇదే గరిష్ఠం, కానీ జీతం ఎక్కువ ఇస్తున్నాం"
>
> Offer 2 ₹70,000 ఎక్కువ. అను దాన్ని తీసుకుంది.
>
> **రెండేళ్ళ తర్వాత:** B1 నుంచి B2 కి promotion వచ్చింది (18 నెలల tenure + cycle). ఇప్పుడు ఆమె B2. అదే సమయంలో offer 1 తీసుకున్నవాళ్ళు B3 (Technical Lead) కి చేరుకున్నారు. అను ఇప్పుడు **ఒక grade వెనుక**, మరియు ఆ grade వల్ల onsite shortlist lo ఆమె పేరు రాలేదు.
>
> ₹70,000 కి ఆమె **రెండు సంవత్సరాలు** అమ్ముకుంది.

నియమం: తక్కువ తేడా ఉన్నప్పుడు (≤10%), **ఎప్పుడూ ఎక్కువ grade తీసుకో**. పెద్ద తేడా ఉన్నప్పుడు (≥25%), డబ్బు తీసుకుని grade ని తర్వాతి switch lo సరిచేసుకో. మధ్యలో ఉంటే – ఆ grade lo promotion cycle ఎంత అని అడిగి నిర్ణయించు.

Key Points

- Grade = promotion timer + onsite అర్హత + role అర్హత + తర్వాతి offer యొక్క ప్రారంభ స్థానం.
- **≤10% తేడా అయితే grade తీసుకో.** ≥25% అయితే డబ్బు తీసుకో.
- Offer letter lo grade code **రాతపూర్వకంగా** ఉండాలి.

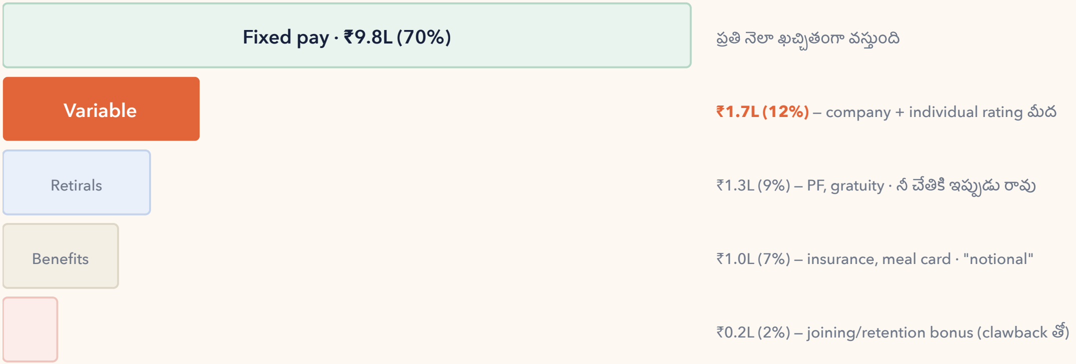
10. CTC structure – fixed, variable, retention; ఏది నిజమైన డబ్బు

వివరణ

Service MNC offer letter lo ఒక పెద్ద సంఖ్య ఉంటుంది – "Total CTC ₹14,00,000". ఆ సంఖ్యలో నీ చేతికి వచ్చేది, వచ్చేలా కనిపించేది, ఎప్పటికీ రానిది – మూడూ కలిసి ఉంటాయి.

₹14L CTC ఒక OFFER LO · నిజంగా ఏది ఎంత

ఒక సాధారణ service MNC CTC విభజన



> **నువ్వు నిజంగా అడగాల్సిన ప్రశ్న**
>
> "CTC ఎంత?" కాదు – **"Fixed pay ఎంత? Variable ఎంత శాతం చెల్లించారు గత రెండేళ్ళలో?"**
> Variable "up to 12%" అంటే – 100% చెల్లించొచ్చు, 40% చెల్లించొచ్చు, 0% కూడా చెల్లించొచ్చు.
> రెండు offers పోల్చేటప్పుడు ఎప్పుడూ **fixed pay** ని పోల్చు, CTC ని కాదు.

ప్రతి భాగం గురించి నిజం

భాగం	నిజం
Fixed pay	నిజమైన డబ్బు. Basic + HRA + special allowance. ఇదే నీ EMI, ఇదే నీ తర్వాతి hike యొక్క ఆధారం
Variable / performance pay	Company performance × individual rating. Service MNCs lo **60-100%** చెల్లించడం సాధారణం, కానీ చెడ్డ సంవత్సరంలో 40% కూడా చూశారు. **హామీ కాదు**
Retirals (PF, gratuity)	నీవే, కానీ ఇప్పుడు కాదు. Gratuity కి **5 సంవత్సరాలు** ఉండాలి – తక్కువ ఉంటే సున్నా
Benefits (insurance, meal card)	విలువ ఉంది కానీ CTC ని ఉబ్బిస్తుంది. వీటిని నగదుగా మార్చలేవు
Joining / retention bonus	సాధారణంగా **clawback** తో – 12-24 నెలల్లో వదిలితే తిరిగి ఇవ్వాలి. CTC ని పెద్దదిగా చూపించడానికి వాడతారు

రెండు offers పోల్చే సరైన పద్ధతి

```
Offer A: CTC ₹14.0L  →  Fixed ₹9.8L  + Variable ₹1.7L + Retirals ₹1.3L + Benefits
₹1.0L + Bonus ₹0.2L
Offer B: CTC ₹13.2L  →  Fixed ₹11.2L + Variable ₹0.7L + Retirals ₹1.0L + Benefits
₹0.3L

CTC ప్రకారం:    A గెలుస్తుంది (₹14.0L vs ₹13.2L)
Fixed ప్రకారం:  B గెలుస్తుంది (₹11.2L vs ₹9.8L) — నెలకి ₹11,600 ఎక్కువ చేతికి
```

> **ఎప్పుడూ ఈ లెక్క వెయ్యి.** CTC అనేది ఒక marketing సంఖ్య. Fixed pay అనేది నీ జీవితం.

Gotchas (సాధారణ తప్పులు)

1. **Variable ని guaranteed అనుకోవడం.** Offer discussion lo "గత రెండు సంవత్సరాల్లో ఈ business unit lo variable payout ఎంత శాతం?" అని అడుగు. సమాధానం దాటవేస్తే, అదే సమాధానం.
2. **Retention bonus clawback ని చదవకపోవడం.** ₹2L joining bonus బాగుంది – కానీ 18 నెలల్లో వదిలితే **పూర్తి మొత్తం, పన్ను కలిపి** తిరిగి ఇవ్వాల్సి రావొచ్చు. ఇది నిన్ను బంధిస్తుంది.
3. **Notice period ని విస్మరించడం.** Service MNCs lo **60-90 రోజులు** సాధారణం. Buy-out ఉందా, ఎంత – ఇది join అయ్యే ముందే తెలియాలి.
4. **Location allowance ని మర్చిపోవడం.** Bangalore/Mumbai vs Hyderabad/Pune vs tier-2 – HRA మరియు location allowance తేడా పెద్దది. Offer ని city తో కలిపి చూడు.

Key Points

- **Fixed pay మాత్రమే నిజమైన, ఖచ్చితమైన డబ్బు.** మిగతాదంతా షరతులతో కూడినది.
- Offers పోల్చేటప్పుడు **CTC కాదు, fixed pay** పోల్చు.
- Variable payout చరిత్రని **అడిగి** తెలుసుకో – అది సాధారణ ప్రశ్న, దురుసుతనం కాదు.
- Joining bonus = clawback తో కూడిన బంధం. కళ్ళు తెరిచి తీసుకో.

Part 2 – నిజమైన Tech Stack

11. React vs Angular – Indian services lo వాస్తవం

వివరణ

Internet lo ఈ చర్చ "ఏది మంచిది" అని జరుగుతుంది. నీ career దృష్ట్యా సరైన ప్రశ్న అది కాదు – **"ఏ ఉద్యోగాలు ఎక్కడ ఉన్నాయి, మరియు నేను ఎక్కడ ఇరుక్కుపోతాను"** అనేది.

NAUKRI OPENINGS (2026) – ముడి సంఖ్యల తేడా

React · ~58,000 openings — React openings దాదాపు 3× ఎక్కువ

Angular · ~19,000 — కానీ ఇది పూర్తి కథ కాదు ↓

React బలంగా ఉన్న చోటు

కొత్త digital transformation projects
US/startup clients, product-తరహా accounts
Greenfield builds, design-system పని
3-5 సం. median: ₹8–14 LPA

Angular బలంగా ఉన్న చోటు

BFSI – banks, insurance, capital markets
Telco, European bank GCCs
Microsoft-stack shops, దీర్ఘకాలిక accounts
3-5 సం. median: ₹7–12 LPA

Service company lo నిజమైన నియమం

నీ stack ని **client నిర్ణయిస్తాడు**, నువ్వు కాదు. React వచ్చినా Angular project lo వెయ్యొచ్చు.
అందుకే allocation కి ముందు "ఏ stack?" అని అడగడం – ఇది నీ కెరీర్ ప్రశ్న, ఉత్సుకత కాదు.

2026 lo framework ఎంపిక ఎక్కువగా **context** మీద ఆధారపడుతుంది – raw performance ఇక తేడా కాదు (React initial render ~200–400ms, Angular తో పోల్చదగినది). Angular పెద్ద teams కి enforced architecture, TypeScript క్రమశిక్షణ ఇస్తుంది; React hiring supply, flexibility ఇస్తుంది.

నీ నిర్ణయానికి ఆచరణాత్మక మార్గదర్శి

- నువ్వు **ఇప్పటికే React** – దాన్నే లోతుగా చెయ్యి. Openings 3× ఎక్కువ, pay కొంచెం ఎక్కువ. Angular ని "చదివి తెలుసు" స్థాయిలో ఉంచు.
- **BFSI account lo Angular project** వచ్చిందా? తిరస్కరించొద్దు. BFSI domain knowledge చాలా విలువైనది, మరియు Angular + TypeScript నేర్చుకోవడం నీ React ని దెబ్బతీయదు – TypeScript, RxJS, DI నమూనాలు నీకు మంచి engineer ని చేస్తాయి.
- **మూడేళ్ళు ఒకే framework lo ఉండటం ప్రమాదం కాదు; మూడేళ్ళు ఒకే రకమైన పని చేయడం ప్రమాదం.** (§48)

Key Points

- React openings ~3× Angular (Naukri, 2026); pay కూడా కొంచెం ఎక్కువ.
- Angular = **BFSI, telco, European GCCs** – ఇవి స్థిరమైన, దీర్ఘకాలిక accounts.
- **Stack ని client నిర్ణయిస్తాడు** – allocation కి ముందే అడుగు.
- Angular project ని తిరస్కరించొద్దు; domain knowledge framework కంటే ఎక్కువ కాలం ఉంటుంది.

---

12. ఒక typical enterprise React project ఎలా ఉంటుంది

వివరణ

Tutorials lo చూసే React app కి, ఒక MNC client project కి మధ్య తేడా చాలా పెద్దది. Interview lo "మీ project architecture ఏమిటి?" అని అడిగినప్పుడు ఈ ఆకారం తెలిస్తే నువ్వు స్పష్టంగా మాట్లాడతావు.

ఒక ENTERPRISE REACT APPLICATION · నిజమైన పొరలు

TYPICAL ENTERPRISE FRONTEND – పొరలు మరియు సాధనాలు

App Shell – routing, auth guard, layout, error boundary, feature flags
React Router · SSO redirect · role-based route guards

Feature modules
accounts / payments
reports / admin

Shared components
Design system wrapper
MUI / AntD / సొంతది

State
Redux Toolkit + RTK Query
లేదా React Query + Context

API layer
Axios interceptors
token refresh, retry

Cross-cutting (ఇవే interview lo అడుగుతారు)
i18n – బహుభాషా clients కి తప్పనిసరి
Accessibility – WCAG 2.2 AA (§16)
Logging / telemetry – AppDynamics, Dynatrace

Build & quality
Vite లేదా Webpack · TypeScript strict
ESLint + Prettier + Husky pre-commit
Jest + React Testing Library · SonarQube

CI/CD – Jenkins లేదా Azure DevOps లేదా GitLab CI
build → lint → test → Sonar quality gate → artifact → DEV → SIT → UAT → PROD

గమనిక: ఇక్కడ **Next.js చాలా తక్కువ**. Enterprise internal apps ఎక్కువగా SPA – SEO అవసరం లేదు, మరియు Node server ని host చేయడానికి client infra approval కావాలి. Next.js ఎక్కువగా public-facing marketing/commerce projects lo.

Tutorials lo లేని, కానీ ప్రతి enterprise project lo ఉండేవి

విషయం	ఎందుకు ఉంటుంది
SSO / SAML / OAuth redirect	Client యొక్క corporate identity provider (Okta, Ping, Azure AD) తో login
Role-based UI	ఒకే screen, user role ని బట్టి వేరే buttons. ఇది backend lo కూడా enforce కావాలి
Session timeout warning	BFSI lo తప్పనిసరి – 15 నిమిషాల idle తర్వాత హెచ్చరిక, తర్వాత logout
Audit trail	ఎవరు ఏమి మార్చారో log చేయడం
Environment config	DEV/SIT/UAT/PROD – API URLs runtime lo inject అవుతాయి, build time lo కాదు
Browser support matrix	క్లయింట్ "IE11 కావాలి" అంటే అది contract. (ఇది తగ్గింది కానీ పూర్తిగా పోలేదు)
Content Security Policy	Security team విధించే headers – inline styles/scripts విరిగిపోతాయి

Interview lo ఈ section ని ఎలా వాడాలి

> **"Tell me about your project architecture" కి మంచి సమాధానం:**
>
> "It's a React SPA for the client's internal claims platform, about 40 screens. We use React with TypeScript in strict mode, Redux Toolkit with RTK Query for server state, and a wrapper layer over Material UI so we can swap the underlying library without touching feature code. Auth is SSO through the client's Azure AD – we hold the token in memory and refresh it through an Axios interceptor rather than localStorage, because the security review flagged XSS exposure. CI is Azure DevOps: lint, unit tests, a SonarQube quality gate at 80% coverage, then promotion through DEV, SIT and UAT. The part I own is the claims-search module and the shared data-table component that four teams use."

> **ఇందులో ఏముంది:** stack, ఒక security నిర్ణయం + **ఎందుకు**, CI pipeline, మరియు **నీ నిర్దిష్ట బాధ్యత**. చివరి వాక్యం చాలా ముఖ్యం – "మా team చేసింది" కాదు, "నేను దీన్ని own చేస్తాను".

Key Points

- Enterprise React = SPA + SSO + role-based UI + i18n + a11y + CI quality gates.
- **Next.js తక్కువ** – internal apps కి SEO అవసరం లేదు, Node hosting కి approval కావాలి.
- Tutorials lo ఎప్పుడూ కనిపించనివి: session timeout, CSP, environment config, audit trail.
- Architecture చెప్పేటప్పుడు ఎప్పుడూ **ఒక నిర్ణయం + ఎందుకు + నీ భాగం** చెప్పు.

---

13. Legacy & migration projects – jQuery, AngularJS, Razor

వివరణ

Service MNC lo నీకు వచ్చే projects lo చాలా భాగం **greenfield కాదు** – అవి పాత systems ని కొత్తవాటికి మార్చే పనులు. "React developer" అని hire అయి, మొదటి ఆరు నెలలు jQuery code చదవడం – ఇది చాలా సాధారణం.

ఇది చెడ్డ వార్త కాదు. **Migration projects lo నేర్చుకునేది ఎక్కువ**, ఎందుకంటే నువ్వు "ఇది ఎందుకు ఇలా ఉంది" అని అర్థం చేసుకోవాలి – greenfield lo ఆ ప్రశ్న ఉండదు.

సాధారణంగా కనిపించే migration రకాలు

నుంచి → కి	ఏం కష్టం
jQuery → React	Global DOM manipulation ని component state కి మార్చడం. చాలా చోట్ల `$('#id')` ఎక్కడి నుంచి వచ్చిందో తెలియదు
AngularJS (1.x) → React/Angular	`$scope`, two-way binding నమూనాలు React lo నేరుగా అనువాదం కావు
.NET MVC Razor → SPA	Server-rendered pages ని API + client rendering గా విడగొట్టడం; session handling పూర్తిగా మారుతుంది

నుంచి → కి	ఏం కష్టం
Class components → Hooks	చిన్నదిగా కనిపిస్తుంది, కానీ lifecycle నుంచి effect కి mapping lo సూక్ష్మమైన bugs
JavaScript → TypeScript	`any` తో మొదలుపెట్టి క్రమంగా strict కి – ఇది నెలల పని

Strangler pattern – migration యొక్క నిజమైన పద్ధతి

ఏ పెద్ద application నీ ఒకేసారి rewrite చేయరు (ఆ ప్రయత్నాలు దాదాపు ఎప్పుడూ విఫలమవుతాయి). బదులుగా **పాత app చుట్టూ కొత్తది నెమ్మదిగా పెరుగుతుంది**:

STRANGLER PATTERN · పాత APP ని ఒకేసారి కాకుండా, ముక్కలుగా మార్చడం

దశ 1 – మొత్తం పాతది

Legacy jQuery app
40 screens

దశ 2 – ఒక route కొత్తది

Legacy
38 screens

React
2

దశ 3 – ఎక్కువ భాగం కొత్తది

Legacy

React
32 screens

ఇది పనిచేయాలంటే కావలసినవి

ఒక **reverse proxy** – ఏ URL పాతదానికి, ఏది కొత్తదానికి వెళ్ళాలో నిర్ణయిస్తుంది
ఒక **ఉమ్మడి session/auth** – user రెండు apps మధ్య తిరిగినా login పోకూడదు

Migration project lo నిజమైన కష్టాలు

పాత code కి tests లేవు – safety net లేదు
Business logic ఎవరికీ పూర్తిగా తెలియదు
"ఇది bug నా feature నా" అని తేల్చలేని ప్రవర్తనలు
రెండు stacks ని ఒకేసారి నిర్వహించడం

కానీ ఇక్కడ నేర్చుకునేవి

పాత code చదవడం – ఇది అరుదైన నైపుణ్యం
Business domain లోతుగా అర్థమవుతుంది
Incremental delivery – నిజమైన engineering
ఇవే interview lo గొప్ప కథలు అవుతాయి

Gotchas (సాధారణ తప్పులు)

1. **"పాత code చెత్త" అని బహిరంగంగా అనడం.** ఆ code రాసినవాళ్ళు ఇంకా team lo ఉండొచ్చు; మరియు ఆ code **పదేళ్ళుగా production lo నడిచింది** – నీ కొత్త code ఇంకా ఒక రోజు కూడా నడవలేదు. వినయం ఇక్కడ వ్యూహం కూడా.
2. **Big-bang rewrite ని ప్రతిపాదించడం.** Junior engineers చేసే అత్యంత సాధారణ తప్పు. Incremental migration ప్రతిపాదించేవాడు senior లా కనిపిస్తాడు.
3. **పాత ప్రవర్తనని పరీక్షించకుండా మార్చడం.** Migration lo నియమం: **కొత్త screen పాత screen లాగే ప్రవర్తించాలి** – మెరుగ్గా కాదు. "నేను ఈ validation ని సరిచేశాను" అనేది migration lo ఒక defect, improvement కాదు (అది వేరే ticket).

Key Points

- Service MNC projects lo migration భాగం **చాలా పెద్దది** – దీనికి సిద్ధపడు.
- **Strangler pattern** = proxy + ఉమ్మడి auth + ఒక్కో route మార్పు.
- Migration lo నియమం: **ప్రవర్తన సమానంగా ఉండాలి**, మెరుగ్గా కాదు.
- పాత code చదవడం ఒక **అరుదైన, విలువైన నైపుణ్యం** – interview lo దీన్ని కథగా వాడు.

14. Micro-frontends & Module Federation – ఎప్పుడు నిజం, ఎప్పుడు resume-driven

వివరణ

Micro-frontend = ఒక పెద్ద UI ని స్వతంత్రంగా build అయ్యి, స్వతంత్రంగా deploy అయ్యే ముక్కలుగా విడగొట్టడం. Webpack **Module Federation** లేదా **single-spa** దీన్ని runtime lo కలుపుతాయి.

Service MNC interviews lo ఇది ఒక fashionable పదంగా మారింది. నిజం ఏమిటంటే – **ఇది కొన్ని చోట్ల నిజమైన అవసరం, చాలా చోట్ల అనవసరమైన సంక్లిష్టత**.

MICRO-FRONTEND · ఎప్పుడు సరైనది, ఎప్పుడు కాదు

నిర్ణయం సాంకేతికం కాదు – అది సంస్థాగతం

ఇవి నిజమైతే – సరైనది ✓

4+ స్వతంత్ర teams, వేరే release cycles
ఒక team deploy చేయాలంటే మిగతావాళ్ళ కోసం ఆగాలి
వేరే vendors / వేరే కంపెనీలు ముక్కలు కడుతున్నాయి
పాత app నుంచి incremental migration (§13)
ఒక platform team ఉంది, shell ని నిర్వహించడానికి

ఇవి నిజమైతే – తప్పు ✗

ఒకే team, 8 మంది, ఒకే release cycle
"Modular గా ఉంటుంది" అనేది ఏకైక కారణం
Shared state ని ముక్కల మధ్య పంచాలి
Design system ఇంకా స్థిరపడలేదు
Platform ని నిర్వహించే మనిషి ఎవరూ లేరు

వచ్చే ఖర్చు – దీన్ని ఎవరూ ముందే చెప్పరు

ఉమ్మడి dependencies (React version) సమన్వయం · cross-app routing · ఉమ్మడి auth · bundle నకిలీ · debugging కష్టం · integration testing ఒక కొత్త సమస్య

Interview lo చెప్పాల్సిన సమాధానం

"Micro-frontends ని **org boundary** ఆధారంగా ఎంచుకోవాలి, tech ప్రాధాన్యత ఆధారంగా కాదు. రెండు teams ఒకే cadence lo release చేస్తుంటే, ఒక monorepo చాలు." – ఇది senior సమాధానం.

Key Points

- Micro-frontend = **సంస్థాగత పరిష్కారం**, సాంకేతిక అలంకారం కాదు.
- నిజమైన trigger: **స్వతంత్ర deploy cycles**. అది లేకపోతే monorepo చాలు.
- ఖర్చు: dependency సమన్వయం, ఉమ్మడి auth, debugging, integration testing.
- Interview lo "ఎప్పుడు వాడకూడదు" అని చెప్పగలిగితే నువ్వు senior లా వినిపిస్తావు.

---

15. Design system & component library పని

వివరణ

పెద్ద accounts lo ఒక **design system** ఉంటుంది – Figma lo designers నిర్వచించిన tokens (రంగులు, spacing, typography) మరియు వాటిని అమలు చేసే React component library.

ఇది frontend engineer కి **రెండు రకాల పని** ఇస్తుంది, మరియు అవి చాలా వేరు:

Design system వాడేవాడు	Design system కట్టేవాడు
Feature screens కడతాడు	Button, Modal, DataTable లాంటి primitives కడతాడు
Library నుంచి import చేస్తాడు	API surface రూపొందిస్తాడు (props, variants, slots)
"ఈ variant లేదు" అని ticket పెడతాడు	Accessibility, keyboard, focus trap, ARIA ని తానే చేయాలి

Design system వాడేవాడు	Design system కట్టేవాడు
2-3 సంవత్సరాల స్థాయి పని	**4+ సంవత్సరాల స్థాయి పని – ఇది promotion మార్గం**

> **వ్యూహాత్మక సలహా:** నీ account lo design system team ఉంటే, అందులోకి వెళ్ళడానికి ప్రయత్నించు. అక్కడ నేర్చుకునేది (API design, accessibility, versioning, ఇతర teams కి support) – అదే నిన్ను **senior** గా మారుస్తుంది, feature screens సంఖ్య కాదు.

Design system పని lo నిజమైన సవాళ్ళు

1. **API surface design.** `<Button variant="primary" size="lg" loading>` – ఈ props ఎంచుకోవడం ఒక ఒప్పందం. తర్వాత మార్చాలంటే 40 teams ని అడగాలి.
2. **Versioning & breaking changes.** ఒక component ni మార్చితే వాడుతున్న ప్రతి app విరిగిపోవచ్చు. Semantic versioning, deprecation warnings, migration guides.
3. **"ఈ ఒక్క case కి exception ఇవ్వండి" ఒత్తిడి.** ఒక team కి ప్రత్యేక variant కావాలి. ప్రతిసారి "సరే" అంటే library ఒక చెత్తకుప్ప అవుతుంది.
4. **Accessibility by default.** Modal lo focus trap, Escape key, `aria-modal`, background scroll lock – ఇవన్నీ **component lo** ఉండాలి, ప్రతి వాడకంలో కాదు.

Key Points

- Design system **వాడటం** vs **కట్టడం** – రెండోది senior-track పని.
- కట్టేవాడికి కావలసినవి: API design, versioning క్రమశిక్షణ, a11y లోతు, "లేదు" చెప్పగలగడం.
- Accessibility component lo ఉండాలి – వాడేవాడి మీద వదలొద్దు.

---

16. Accessibility (WCAG 2.2 AA) – ఇది contract, option కాదు

వివరణ

భారతీయ engineers చాలా మంది accessibility ని "మంచిది, కానీ సమయం ఉంటే" అనుకుంటారు. Enterprise client projects lo **అది తప్పు, మరియు ఖరీదైన తప్పు**.

చాలా client contracts lo **WCAG 2.2 Level AA** ఒక **delivery acceptance criterion** గా ఉంటుంది. అంటే: accessibility audit fail అయితే **release ఆగిపోతుంది**, మరియు అది ఒక commercial సమస్య అవుతుంది – ఒక సాంకేతిక సమస్య కాదు.

ACCESSIBILITY · ఎప్పుడు సరిచేస్తే ఎంత ఖర్చు

"షిఫ్ట్ లెఫ్ట్" – ఇది నినాదం కాదు, ఇది ఖర్చు వక్రరేఖ

Design — రంగు contrast, focus క్రమం – ఇక్కడ సరిచేస్తే ఖర్చు దాదాపు సున్నా

Component build — Semantic HTML, ARIA, keyboard – ఒక component కి ఒక్కసారి

Feature development — ప్రతి screen lo విడిగా సరిచేయాలి

QA / Audit — Audit report → 200 issues → retrofit

Production – ఇక్కడ దొరికితే: release ఆపడం, client escalation, contractual penalty సంభాషణ

WCAG 2.2 AA lo ఎక్కువ సమస్యలు **design system మరియు interaction నమూనాల్లో** పాతుకుపోయి ఉంటాయి – చివర్లో patch చేయలేం.

Frontend engineer గా నువ్వు తెలుసుకోవలసిన కనీసం

విషయం	ఆచరణలో ఏమిటి
Semantic HTML	`<button>` వాడు, `<div onClick>` కాదు. ఇది 60% సమస్యలని తొలగిస్తుంది
Keyboard navigation	ప్రతి interactive element Tab తో చేరుకోగలగాలి, Enter/Space తో పనిచేయాలి
Focus management	Modal తెరిస్తే focus లోపలికి, మూస్తే తిరిగి trigger మీదికి
Visible focus indicator	`outline: none` రాసేముందు ఒకటికి రెండుసార్లు ఆలోచించు
Labels	ప్రతి input కి `<label>` లేదా `aria-label`. Placeholder label కాదు
Colour contrast	Text కి 4.5:1, పెద్ద text కి 3:1
Error messages	`aria-live` తో ప్రకటించాలి; రంగు ఒక్కటే సూచన కాకూడదు
Images	అర్థవంతమైన `alt`; అలంకారం అయితే `alt=""`

Interview lo ఇది ఒక ఆయుధం

చాలా తక్కువ మంది candidates accessibility గురించి నమ్మకంగా మాట్లాడతారు. నువ్వు మాట్లాడితే **వెంటనే వేరుగా కనిపిస్తావు** – ముఖ్యంగా BFSI, government, healthcare accounts lo, ఎందుకంటే అక్కడ ఇది contractual.

ఒక మంచి సమాధానం:

"We treat accessibility as a definition-of-done item, not a QA phase. The biggest win was fixing it in the design system rather than per screen — our Modal handles focus trap, Escape, `aria-modal` and scroll lock internally, so forty feature screens got it for free. The things I check in every PR are semantic elements over div-with-onClick, a visible focus ring, labels tied to inputs, and that errors are announced through an `aria-live` region rather than only turning the border red. We run axe in CI, but automated tools only catch about a third of WCAG issues, so we also do a keyboard-only pass before UAT."

Key Points

- WCAG **2.2 Level AA** అనేది చాలా enterprise contracts lo **acceptance criterion**.
- **Design system lo సరిచేస్తే** ఒక్కసారి; feature lo సరిచేస్తే ప్రతిసారీ.
- కనీసం: semantic HTML, keyboard, focus management, labels, contrast, `aria-live` errors.
- Automated tools WCAG సమస్యల్లో **మూడో వంతు మాత్రమే** పట్టుకుంటాయి — keyboard pass తప్పనిసరి.
- Interview lo ఇది **అరుదైన, బలమైన** వేరుపాటు.

---

17. Java/Spring backend — frontend dev కి ఎంత తెలియాలి

వివరణ

భారతీయ service MNCs lo backend యొక్క **అత్యంత సాధారణ stack Java/Spring Boot** — Node.js కాదు. నువ్వు MERN నేపథ్యం నుంచి వస్తే ఇది ఒక ఆశ్చర్యం కావొచ్చు, మరియు ఇది ఒక ముఖ్యమైన వ్యూహాత్మక విషయం:

> **జాగ్రత్త:** చాలా service MNC **"Full Stack Developer"** requisitions అంటే **Java/Spring + Angular/React** — MERN కాదు. నువ్వు MERN నేపథ్యం అయితే, **"Frontend Developer (React)"** లేదా **"UI Developer"** requisitions కి apply చెయ్యి, మరియు recruiter screen lo backend stack ని **ముందే** నిర్ధారించుకో. ఇది ఒక వారం వృథా అవ్వకుండా కాపాడుతుంది.

Frontend dev కి అవసరమైన Java/Spring కనీసం

నీకు Java రాయడం రానవసరం లేదు. కానీ ఈ ఆరు విషయాలు తెలిస్తే నువ్వు backend team తో సమానంగా మాట్లాడతావు — మరియు అదే నిన్ను "కేవలం UI వాడు" నుంచి "engineer" గా మారుస్తుంది:

విషయం	నీకు ఎందుకు కావాలి
REST controller ఆకారం	`@GetMapping("/api/claims")` — endpoint ఎక్కడ నిర్వచించబడిందో చదవగలగాలి
DTO vs entity	API response ఆకారం ఎందుకు DB table లా లేదో అర్థమవుతుంది
HTTP status codes	400 vs 422 vs 500 — ఏది నీ తప్పు, ఏది వాళ్ళది

విషయం	నీకు ఎందుకు కావాలి
Pagination నమూనా	`page/size` vs `offset/limit` vs cursor – infinite scroll దీని మీదే ఆధారపడుతుంది
Auth flow	JWT ఎక్కడ validate అవుతుంది, refresh token ఎలా పనిచేస్తుంది
CORS & proxy	Local dev lo CORS error వస్తే – proxy config తో పరిష్కరించాలా, backend header మార్చాలా

API contract – ఇక్కడే frontend-backend గొడవలు జరుగుతాయి

> **నిజమైన పరిస్థితి:** Sprint మొదట్లో backend "API 3 రోజుల్లో ఇస్తాం" అంటారు. నువ్వు ఎదురుచూస్తావు. 6వ రోజు API వస్తుంది, field పేర్లు వేరుగా ఉంటాయి. ఇప్పుడు నీ దగ్గర 2 రోజులు ఉన్నాయి.
>
> **పరిష్కారం:** Sprint మొదట్లోనే **contract ని రాతపూర్వకంగా** ఖరారు చేసుకో – ఒక sample JSON response. దాన్ని `mocks/` folder lo పెట్టి MSW (Mock Service Worker) తో నీ UI ని సమాంతరంగా కట్టు. API వచ్చాక కేవలం switch.
>
> ఇది చేసేవాడు sprint ని కాపాడతాడు. **Interview lo ఇది ఒక అద్భుతమైన కథ.**

Key Points

- Service MNC backend = **ఎక్కువగా Java/Spring Boot**. "Full Stack" అంటే తరచుగా Java + React.
- నీకు Java రాయడం అవసరం లేదు, కానీ **API contract, status codes, auth flow, pagination** తెలియాలి.
- **Contract-first + mocks** – ఇది sprint ని కాపాడే ఆచరణ, మరియు గొప్ప interview కథ.

---

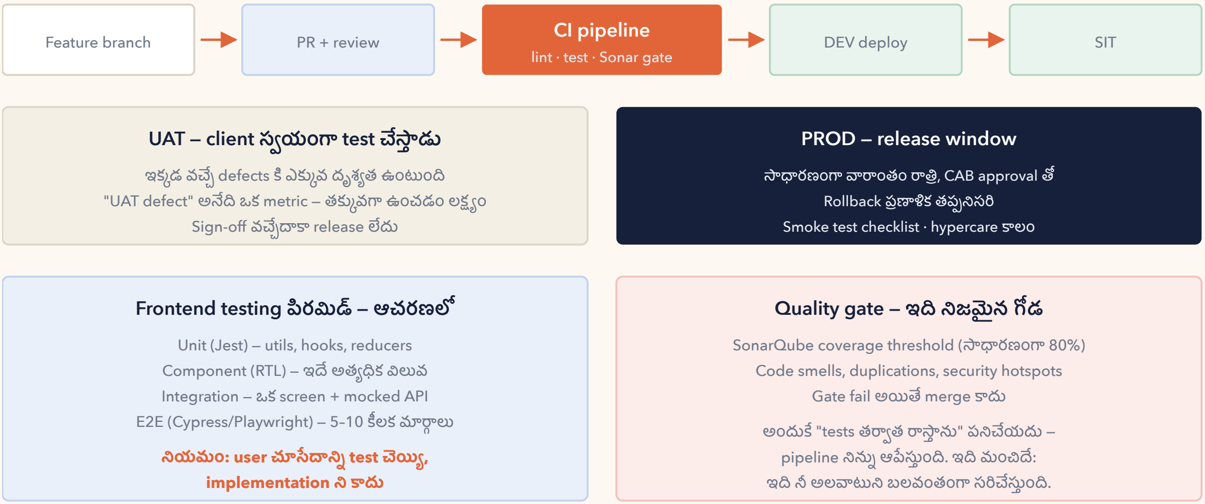
18. Testing, CI/CD, release process

వివరణ

MNC lo "నేను code రాశాను, పని చేస్తోంది" అనేది సగం పని. మిగతా సగం – అది pipeline ద్వారా ప్రవహించి, నాలుగు environments గుండా వెళ్ళి, production కి చేరడం.

CODE నుంచి PRODUCTION వరకు · ఒక ENTERPRISE RELEASE మార్గం

ఏ PR నుంచి client వాడకందారు వరకు



Environments – ఏది దేనికి

Environment	ఎవరు వాడతారు	దేనికి
DEV	Developers	రోజువారీ integration; తరచుగా విరిగిపోతుంది, పర్వాలేదు
SIT (System Integration Test)	QA team	అన్ని systems కలిపి పనిచేస్తున్నాయా
UAT (User Acceptance Test)	**Client users**	వ్యాపార దృష్టితో ఆమోదం. ఇక్కడ defect = దృశ్యత
PROD	నిజమైన వాడకందారులు	Release window, CAB approval, rollback plan

Key Points

- **Quality gate** (Sonar coverage) merge ని ఆపగలదు – tests ఐచ్ఛికం కాదు.
- **RTL component tests** frontend lo అత్యధిక విలువ ఇస్తాయి; E2E ని 5-10 కీలక మార్గాలకి పరిమితం చెయ్యి.
- **UAT defect** ఒక metric – దాన్ని తక్కువగా ఉంచడం నీ కనిపించే నాణ్యత.
- PROD release = window + approval + rollback plan + hypercare. ఇది ప్రక్రియ, ఒక `git push` కాదు.

---

19. Service company lo నువ్వు చేయనివి

వివరణ

ఇది కూడా అంతే ముఖ్యం – ఎందుకంటే ఇక్కడే చాలా మంది నిరాశ పడతారు. Product company నుంచి వచ్చినవాళ్ళు, లేదా YouTube చూసి అంచనాలు పెంచుకున్నవాళ్ళు ఈ గోడకి కొట్టుకుంటారు.

నువ్వు సాధారణంగా చేయనిది	ఎందుకు
Tech stack ఎంచుకోవడం	Client architecture team నిర్ణయిస్తుంది; కొన్నిసార్లు contract lo రాసి ఉంటుంది
Product నిర్ణయాలు	"ఈ feature అవసరం లేదు" – ఇది client business team పరిధి
Infrastructure	AWS/Azure ని client platform team నిర్వహిస్తుంది. నీకు console access కూడా ఉండకపోవచ్చు
Production లోకి నేరుగా deploy	Release process, CAB approval, change ticket – ఒక్క క్లిక్ కాదు
Open source contribution (ఆ code నుంచి)	Client IP. Code బయటికి వెళ్ళదు, portfolio lo చూపించలేవు
కొత్త library ని స్వేచ్ఛగా చేర్చడం	Security scan, license approval, architecture review కావాలి

ఇది నిరుత్సాహపరిచే జాబితా – కానీ దీనికి ఒక ప్రకాశవంతమైన వైపు ఉంది. ఈ పరిమితులే నిన్ను **ఒక నిర్దిష్ట రకమైన engineer** గా తయారుచేస్తాయి: పరిమితుల్లో పనిచేసేవాడు, ఇతరుల code చదవగలిగేవాడు, ప్రక్రియని గౌరవించేవాడు, మరియు client భాషలో మాట్లాడగలిగేవాడు. ఇవి product companies lo **అరుదైన** లక్షణాలు – మరియు నీ exit interview lo (§45) ఇవే నీ బలం.

అయితే ఈ పరిమితుల్లో ఎలా పెరగాలి

1. **నీ పరిధిలో నాణ్యతని పెంచు.** Stack మార్చలేవు, కానీ నీ component ని బాగా రాయగలవు, tests రాయగలవు, a11y చేయగలవు.
2. **Internal POC / hackathon.** చాలా MNCs lo ఇవి ఉంటాయి. ఇక్కడ నువ్వు స్వేచ్ఛగా కొత్తది వాడొచ్చు, మరియు ఇవి appraisal lo లెక్కలోకి వస్తాయి.
3. **సొంత side project.** Client code బయటికి రాదు కాబట్టి, **నీ GitHub నీదే** – తర్వాతి interview lo చూపించడానికి ఇది అవసరం.
4. **Client కి సూచనలు ఇవ్వడం – సరైన మార్గంలో.** "ఇది మారుద్దాం" అనడం కాదు; "ఈ approach ఈ సమస్యని పరిష్కరిస్తుంది, ఖర్చు ఇది" అని Tech Lead ద్వారా ప్రతిపాదించడం. ఇలా చేసేవాళ్ళు గమనించబడతారు.

Key Points

- Stack, product నిర్ణయాలు, infra, deploy – ఇవి సాధారణంగా నీ పరిధిలో ఉండవు.
- **Client code నీ portfolio కాదు** – సొంత GitHub నిర్వహించు.
- పరిమితుల్లో పెరగడానికి: నీ పరిధిలో నాణ్యత + internal POC + సొంత projects + సరైన మార్గంలో ప్రతిపాదన.

Part 3 – ఒక రోజు, ఒక sprint, ఒక సంవత్సరం

20. రోజువారీ – standup, tickets, PR review, timesheet

వివరణ

ఒక సాధారణ రోజు · OFFSHORE (INDIA) TEAM

IST TIMELINE – US client account

9:30 – లోపలికి	Email, overnight client replies, build status చూడటం
10:00 – Standup	15 నిమిషాలు · నిన్న / ఈరోజు / blockers
10:15–13:00	**లోతైన పని** – ఇదే నీ అత్యుత్తమ సమయం, దీన్ని కాపాడుకో
14:00–16:00	PR reviews, grooming, QA తో defect చర్చ
16:00–18:00	Client overlap మొదలు – calls, status, clarifications
18:30 – ముగింపు	Timesheet · రేపటి ప్రణాళిక · handover note

గమనిక: European clients కి overlap ఉదయం ఉంటుంది (13:00 IST నుంచి) – అప్పుడు లోతైన పని మధ్యాహ్నం కాదు, ఉదయం.

Standup – 15 నిమిషాల్లో నీ ఖ్యాతి

Standup ఒక ఆచారంలా అనిపిస్తుంది, కానీ **నీ manager నిన్ను చూసే ప్రధాన కిటికీ ఇదే**. రోజుకి 90 సెకన్లు × 250 రోజులు = నీ మీద ఏర్పడే అభిప్రాయం.

చెడ్డ standup	మంచి standup
"నిన్న claims screen మీద పనిచేశాను. ఈరోజు కూడా అదే. Blockers లేవు."	"నిన్న claims search UI పూర్తయింది, filters పనిచేస్తున్నాయి. ఈరోజు pagination + error states – సాయంత్రానికి PR. Blocker: search API 500 ఇస్తోంది 90 రోజుల range కి; Ramesh తో 11:00 కి చూస్తున్నాం."
"దాదాపు అయిపోయింది" (మూడు రోజులుగా)	"80% – మిగిలింది error handling. రేపు మధ్యాహ్నం."
Blocker ని దాచడం	Blocker ని **మొదటి రోజే** చెప్పడం

> **నియమం:** "దాదాపు అయిపోయింది" అనే మాట standup lo నిషేధం. దాని బదులు **ఏది అయ్యింది, ఏది మిగిలింది, ఎప్పటికి** – మూడూ చెప్పు.

PR review – ఇవ్వడం మరియు తీసుకోవడం

నీ PR ని ఎలా పెట్టాలి:

- Description lo: *ఏమి మార్చావు, ఎందుకు, ఎలా test చేశావు*, మరియు screenshot (UI అయితే).
- చిన్నవిగా ఉంచు. 800 lines PR ని ఎవరూ సరిగ్గా review చేయరు – "LGTM" వస్తుంది, అది review కాదు.
- నీకే తెలిసిన సందేహాలు ఉంటే PR lo comment పెట్టు: *"ఈ భాగంలో memo అవసరమా అని నాకు ఖచ్చితంగా తెలియదు – అభిప్రాయం?"*

ఇతరుల PR ని ఎలా review చేయాలి:

- **24 గంటల్లో** చూడు. Review ఆలస్యం = sprint ఆలస్యం.
- సూచనలని ప్రశ్నలుగా అడుగు: *"ఇక్కడ `useMemo` లేకపోతే ఏమవుతుంది?"* – *"useMemo వాడు"* కంటే ఇది మెరుగు.
- చిన్న విషయాలని `nit:` అని గుర్తుపెట్టు, పెద్దవాటిని వేరుగా చూపించు.

Key Points

- **Standup = నీ దృశ్యత.** 90 సెకన్లలో: ఏమయ్యింది, ఏమవుతోంది, ఏది ఆపుతోంది.
- "దాదాపు అయిపోయింది" నిషేధం – శాతం + మిగిలినది + తేదీ చెప్పు.
- PR: చిన్నది, వివరణతో, screenshot తో. Review: **24 గంటల్లో**, ప్రశ్నలుగా.
- **Timesheet రోజూ నింపు** – వారాంతంలో కాదు (§2).

---

21. Sprint ceremonies – planning, grooming, demo, retro

వివరణ

రెండు వారాల sprint అనేది చాలా accounts lo ప్రామాణికం. ఈ నాలుగు meetings lo ఏమి జరుగుతుందో, మరియు **నువ్వు ఎక్కడ విలువ చూపించాలో** తెలియడం ముఖ్యం.

Ceremony	ఎప్పుడు	నీ నిజమైన పని ఇక్కడ
Grooming / Refinement	Sprint మధ్యలో, తర్వాతి sprint కోసం	**ప్రశ్నలు అడగడం.** ఇదే అత్యధిక విలువ ఉన్న ceremony
Sprint Planning	Sprint మొదటి రోజు	Estimate ఇవ్వడం, commitment తీసుకోవడం
Daily Standup	రోజూ 15 నిమి	దృశ్యత, blockers
Sprint Review / Demo	చివరి రోజు	క్లయింట్ ముందు నీ పని చూపించడం
Retrospective	Demo తర్వాత	ప్రక్రియని మెరుగుపరచడం

Grooming lo అడగాల్సిన ప్రశ్నలు – ఇదే నిన్ను వేరు చేస్తుంది

ఒక story: *"As a user, I want to export my transactions to Excel."* – junior "సరే" అంటాడు. నువ్వు ఇవి అడగాలి:

1. **పరిమాణం:** "ఎన్ని rows గరిష్ఠంగా? 10 లక్షలుంటే browser lo generate చేయలేం – server-side కావాలి."
2. **Format:** "నిజంగా .xlsx నా, CSV చాలా? xlsx కి ఒక library కావాలి, bundle 200KB పెరుగుతుంది."
3. **Async నా sync నా:** "పెద్ద export అయితే – వెంటనే download నా, email link నా?"
4. **అనుమతులు:** "అన్ని roles కి export ఉందా? Audit log కావాలా?"
5. **Empty/error:** "డేటా లేకపోతే? Export విఫలమైతే?"

> ఈ ఐదు ప్రశ్నలు అడిగిన రోజున – **BA, Tech Lead, PM ముగ్గురికీ** నువ్వు గుర్తుండిపోతావు. మరియు అవి sprint lo 3 రోజుల re-work ని ఆపుతాయి.

Story points – ఒక తరచుగా అపార్థం చేసుకునే విషయం

Story points **సమయం కాదు** – అవి **సాపేక్ష సంక్లిష్టత**. ఒక 5-point story 8-point story కంటే సులభం; కానీ "5 points = 5 రోజులు" అని అనువదించడం తప్పు (అయినా చాలా teams అలాగే చేస్తాయి).

Points	సాధారణంగా అర్థం
1	చాలా చిన్నది – ఒక label మార్పు, ఒక config
2–3	సాధారణ feature, తెలిసిన నమూనా
5	కొంత అనిశ్చితి, లేదా బహుళ files
8	పెద్దది – **విడగొట్టడం గురించి ఆలోచించు**
13+	ఇది story కాదు, ఇది epic. తప్పకుండా విడగొట్టు

Retro lo ఎలా మాట్లాడాలి

Retro lo వ్యక్తులని నిందించడం విషం. **వ్యవస్థని** విమర్శించు:

- ❌ "Backend team ఆలస్యంగా API ఇచ్చింది"
- ✓ "ఈ sprint lo మేము API కోసం 3 రోజులు ఆగాం. **వచ్చే sprint lo contract ని మొదటి రోజే ఖరారు చేసి mocks తో మొదలుపెడదాం**" – ఇది ఒక పరిష్కారంతో కూడిన విమర్శ.

Key Points

- **Grooming అత్యధిక విలువ ఉన్న ceremony** – అక్కడ ప్రశ్నలు అడగడమే నీ సీనియారిటీ సంకేతం.
- Story points = సాపేక్ష సంక్లిష్టత, రోజులు కాదు. **8+ అంటే విడగొట్టు.**
- Retro lo వ్యక్తులని కాదు, **ప్రక్రియని** విమర్శించు – మరియు ఎప్పుడూ ఒక పరిష్కారంతో.

---

22. SAFe / PI planning – పెద్ద accounts lo

వివరణ

ఒక account lo 5–10 scrum teams ఉంటే, సాధారణ Scrum సరిపోదు – teams ఒకదానిపై ఒకటి ఆధారపడతాయి. అప్పుడు **SAFe (Scaled Agile Framework)** వాడతారు. ఇది చాలా పెద్ద MNC accounts lo ఉంటుంది, మరియు interview lo "మీరు SAFe follow చేశారా?" అని అడగడం సాధారణం.

SAFE · ఒక PROGRAM INCREMENT (PI) ఎలా ఉంటుంది

PI = 5 sprints (సుమారు 10-12 వారాలు)

PI Planning 2 రోజులు · అన్ని teams	Sprint 1	Sprint 2	Sprint 3	Sprint 4	IP Sprint innovation + planning

PI Planning lo ఏం జరుగుతుంది

అన్ని teams ఒకే గదిలో (లేదా call lo)
తర్వాత 10 వారాల features ని ప్రణాళిక వేస్తారు
Team-to-team dependencies ని బహిరంగంగా గుర్తిస్తారు
ప్రతి team తన objectives కి commit అవుతుంది

నీకు ఇక్కడ ఉన్న అవకాశం

Dependency ని ముందే చెప్పడం – "ఈ feature కి payments team API కావాలి, అది sprint 3 lo వస్తుందా?"

ఇది అడిగేవాళ్ళు తక్కువ. అడిగితే నువ్వు సీనియర్ గా గుర్తించబడతావు.

SAFe గురించి నిజాయితీ

ఇది చాలా meetings, చాలా documentation తెస్తుంది – మరియు చాలా engineers దీన్ని ఇష్టపడరు.
కానీ interview lo "SAFe lo పనిచేశాను, PI planning lo dependencies ఎలా నిర్వహించామో తెలుసు" అనేది **బలమైన సంకేతం** – ఇది పెద్ద account అనుభవాన్ని సూచిస్తుంది.

Key Points

- SAFe = బహుళ teams ని సమన్వయం చేసే framework; **PI = ~5 sprints**.
- PI planning lo నీ అవకాశం: **dependencies ని ముందే బహిరంగంగా చెప్పడం**.
- Interview lo SAFe అనుభవం = **పెద్ద account** అనుభవం అని సంకేతం.

---

23. Client తో మాట్లాడటం – status call, demo, defect triage, UAT

వివరణ

Product company lo నువ్వు client ని చూడవు. Service company lo **నువ్వే client ముఖం**. ఇదే ఈ ఉద్యోగంలో అతిపెద్ద తేడా, మరియు interviews lo communication కి అంత బరువు ఎందుకు ఉందో దీనికి కారణం.

నాలుగు రకాల client సంభాషణలు

రకం	నీ లక్ష్యం	సాధారణ తప్పు
Status call	స్పష్టత ఇవ్వడం, ఆశ్చర్యాలు లేకుండా చేయడం	సమస్యని దాచడం – తర్వాత అది పెద్దదవుతుంది
Demo	పూర్తయినది నమ్మకంగా చూపించడం	పూర్తికాని దాన్ని చూపించి "ఇది ఇంకా..." అనడం
Defect triage	కారణాన్ని త్వరగా, నిందలు లేకుండా తేల్చడం	"అది backend సమస్య" అని బహిరంగంగా అనడం
UAT support	Client user కి సహాయం చేయడం	సాంకేతిక పరిభాషలో వివరించడం

Demo – ఒక ప్రత్యేకమైన నైపుణ్యం

Demo lo ఒక్క నిమిషంలో నీ రెండు వారాల పని అంచనా వేయబడుతుంది. కొన్ని ఆచరణాత్మక నియమాలు:

1. **ముందుగా ఒకసారి నడిపి చూడు** – అదే data తో, అదే environment lo. "డెమోలో మాత్రమే వచ్చే bug" ఒక క్లాసిక్.
2. **కథలా చెప్పు, feature list లా కాదు.** *"ఒక claims agent ఒక పాత claim వెతకాలనుకుంటే..."* అని మొదలుపెట్టు.

3. **పూర్తికానిదాన్ని చూపించొద్దు.** చూపించాల్సి వస్తే, ముందే స్పష్టంగా చెప్పు: *"This is behind a feature flag and not yet wired to the live API — I'm showing the interaction, not the data."*
4. **సాంకేతిక వివరాలు వద్దు.** Client కి "Redux Toolkit వాడాం" అనవసరం. "Filter ఇప్పుడు 2 సెకన్లలో వస్తోంది, ముందు 8" — ఇది వాళ్ళ భాష.

Defect triage — ఒక రాజకీయ క్షణం

Production lo ఒక bug. Client call lo అందరూ ఉన్నారు. ఇది frontend దా backend దా data దా అని తెలియదు.

> **చెడ్డ సమాధానం:** "That's a backend issue, the API is returning wrong data."
> *(నిజమైనా — ఇది రక్షణాత్మకంగా, జట్టు వ్యతిరేకంగా వినిపిస్తుంది.)*
>
> **మంచి సమాధానం:** "Let me confirm what we're seeing. The UI is rendering exactly what it receives — I can see the response in the network tab has the closed claims included. So the filter isn't being applied somewhere upstream. Ramesh, can we check the query on your side? Meanwhile I'll add a guard on the UI so users don't see closed items even if they come through, and we can take that out once the root cause is fixed."

> **తేడా ఏమిటి:** రెండోది — సాక్ష్యం చూపిస్తుంది, ఒక తాత్కాలిక పరిష్కారం ప్రతిపాదిస్తుంది, మరియు **జట్టుగా మాట్లాడుతుంది**. Client కి కావలసింది నింద కాదు, పరిష్కారం.

Key Points

- Service company lo **నువ్వే client ముఖం** — ఇదే communication కి అంత బరువు ఉండటానికి కారణం.
- Demo: ముందే నడిపి చూడు, కథలా చెప్పు, **సాంకేతిక పరిభాష వద్దు**.
- Defect triage lo: సాక్ష్యం + తాత్కాలిక పరిష్కారం + జట్టు భాష. **ఎప్పుడూ బహిరంగంగా నిందించొద్దు.**
- చెడ్డ వార్తని **ముందే** చెప్పు. ఆలస్యంగా వచ్చిన చెడ్డ వార్త రెండింతలు చెడ్డది.

---

24. Onshore-offshore model — timezone, handover, "overlap hours"

వివరణ

చాలా service MNC projects **onshore-offshore** నమూనాలో నడుస్తాయి: కొంతమంది client దేశంలో (onshore), ఎక్కువమంది భారత్‌లో (offshore). ఈ నిర్మాణం నీ పనివేళలని, నీ దృశ్యతని, మరియు నీ onsite అవకాశాలని ప్రభావితం చేస్తుంది.

Client ప్రాంతం	IST overlap	నీ రోజు ఎలా ఉంటుంది
US East (EST)	~18:00–21:00 IST	సాయంత్రం calls; లోతైన పని ఉదయం
US West (PST)	~21:00–23:00 IST	ఆలస్యమైన calls — అత్యంత కష్టమైనది
UK (GMT)	~14:00–18:00 IST	సౌకర్యవంతం

Client ప్రాంతం	IST overlap	నీ రోజు ఎలా ఉంటుంది
Europe (CET)	~13:00–17:00 IST	సౌకర్యవంతం
APAC (SG/AU)	~7:00–13:00 IST	ఉదయం calls

> **Offer తీసుకునే ముందు ఇది అడుగు.** "What's the client location and the expected overlap window?" – US West account అంటే నీ రోజువారీ జీవితం చాలా వేరుగా ఉంటుంది. ఇది జీతం కంటే ఎక్కువ కాలం నీ మీద ప్రభావం చూపిస్తుంది.

Handover note – ఒక చిన్న అలవాటు, పెద్ద ప్రతిఫలం

Offshore team రోజు ముగించేటప్పుడు onshore team రోజు మొదలవుతుంది. ఒక మంచి handover note:

```
EOD 14-Mar – Claims Search

అయినవి:
  • Filter UI + date range validation – PR #482 (review కోసం ఎదురుచూస్తోంది)
  • CLM-1203 defect fix – SIT lo verify చేశాను

జరుగుతున్నవి:
  • Pagination – 70%; API cursor format మీద స్పష్టత కావాలి (Ramesh కి అడిగాను)

మీకు కావలసినవి (onshore):
  • PR #482 review – ఇది merge అయితే రేపు నేను SIT deploy చేయగలను
  • Client నుంచి: export format .xlsx నా .csv నా – grooming lo తేలలేదు

Blockers: లేవు
```

ఇది రాయడానికి 4 నిమిషాలు. దీని వల్ల వచ్చేది – onshore team నీ పనిని **చూస్తుంది**, మరియు నిన్ను నమ్మదగినవాడిగా గుర్తిస్తుంది. §42 lo onsite ఎంపిక ఎలా జరుగుతుందో చూస్తే ఇది ఎంత ముఖ్యమో అర్థమవుతుంది.

Key Points

- Client timezone నీ రోజువారీ జీవితాన్ని నిర్ణయిస్తుంది – **offer కి ముందే అడుగు**.
- **Overlap hours** lo calls, మిగతా సమయంలో లోతైన పని – ఈ నిర్మాణాన్ని ఉద్దేశపూర్వకంగా వాడుకో.
- **EOD handover note** – 4 నిమిషాల అలవాటు, onshore దృశ్యత కోసం అత్యధిక ప్రతిఫలం.

---

25. Production support – L2/L3 rotation, RCA

వివరణ

చాలా accounts lo development team **production support బాధ్యత కూడా** తీసుకుంటుంది. దీన్ని "on-call" లేదా "L3 support" అంటారు, మరియు సాధారణంగా ఒక rotation ఉంటుంది – వారానికి ఒకరు.

స్థాయి	ఎవరు	ఏం చేస్తారు
L1	Service desk	Ticket తీసుకోవడం, తెలిసిన పరిష్కారాలు (password reset లాంటివి)

స్థాయి	ఎవరు	ఏం చేస్తారు
L2	Support team	Known issues, config, restart, runbook అనుసరించడం
L3	**Dev team – నువ్వు**	Code స్థాయి సమస్యలు, root cause, fix

Support week lo ఏమి జరుగుతుంది

- **Ticket వస్తుంది** – priority తో (P1 = production down, P2 = పెద్ద ప్రభావం, P3/P4 = చిన్నవి).
- **P1 కి SLA** ఉంటుంది – ఉదాహరణకి "15 నిమిషాల్లో స్పందన, 4 గంటల్లో పరిష్కారం". ఇది contract lo ఉంటుంది.
- నువ్వు **reproduce** చేయాలి, **root cause** కనిపెట్టాలి, **fix** లేదా **workaround** ఇవ్వాలి.
- తర్వాత **RCA document** రాయాలి.

RCA (Root Cause Analysis) – ఒక ముఖ్యమైన document

RCA ఒక formality కాదు; ఇది client కి వెళ్తుంది, మరియు **నీ పేరు దాని మీద ఉంటుంది**. మంచి RCA నిర్మాణం:

```
1. ఏమి జరిగింది        – user ఏమి చూశాడు, ఎప్పటి నుంచి ఎప్పటిదాకా, ఎంతమందికి
2. ప్రభావం             – 340 users, 2 గంటలు, 12 transactions విఫలం
3. Root cause          – సాంకేతిక కారణం, ఒక్క వాక్యంలో
4. ఎలా గుర్తించాం      – alert నా, user complaint నా? (user complaint అయితే అది కూడా ఒక సమస్య)
5. తక్షణ చర్య          – ఏమి చేసి పరిష్కరించాం
6. శాశ్వత పరిష్కారం     – code fix, test, monitoring
7. నివారణ              – ఇది మళ్ళీ జరగకుండా ఏం మారుస్తున్నాం
```

> **7వ అంశం అత్యంత ముఖ్యం.** "Developer జాగ్రత్తగా ఉంటాడు" అనేది నివారణ కాదు. "ఈ case కి ఒక unit test చేర్చాం, మరియు bundle size కి CI check పెట్టాం" – ఇది నివారణ.

Support rotation గురించి నిజాయితీ

- ఇది **అలసటనిచ్చే పని** – ముఖ్యంగా US account lo రాత్రి calls వస్తే.
- కానీ ఇక్కడ నేర్చుకునేది చాలా: production debugging, logs చదవడం, monitoring tools, ఒత్తిడిలో ప్రశాంతంగా ఉండటం.
- **Interview lo ఇది బలమైన కథ**: "P1 incident lo నేను root cause కనిపెట్టి 40 నిమిషాల్లో workaround ఇచ్చాను" – ఇది చాలా candidates దగ్గర ఉండదు.

Key Points

- Dev team సాధారణంగా **L3** – code స్థాయి production సమస్యలు.
- **P1 కి SLA contract lo** ఉంటుంది – ఇది వ్యాపార విషయం, సాంకేతికం మాత్రమే కాదు.
- **RCA lo 7వ అంశం (నివారణ)** అత్యంత ముఖ్యం; "జాగ్రత్తగా ఉంటాం" నివారణ కాదు.
- Support అనుభవం = interview lo అరుదైన, బలమైన కథ.

26. Appraisal cycle – rating నిజంగా ఎలా వస్తుంది

వివరణ

ఇది బహుశా ఈ Part lo అత్యంత ముఖ్యమైన section. చాలా మంది "నేను బాగా పనిచేశాను కాబట్టి మంచి rating వస్తుంది" అనుకుంటారు – మరియు ఆశ్చర్యపోతారు.

APPRAISAL · నీ RATING నిజంగా ఎక్కడ నిర్ణయమవుతుంది

నువ్వు అనుకునేది vs నిజంగా జరిగేది

నువ్వు అనుకునేది

నేను బాగా పనిచేశాను
→ manager చూశాడు
→ మంచి rating వస్తుంది

(ఒక సరళ రేఖ)

నిజంగా జరిగేది

నీ self-appraisal → manager యొక్క input
→ PM/client feedback → **calibration meeting**
→ bell curve పరిమితి → rating

(ఐదు అడుగులు, ఒకటి మాత్రమే నీది)

Calibration meeting – ఇక్కడే అసలు నిర్ణయం జరుగుతుంది

అన్ని managers ఒక గదిలో కూర్చుని, తమ team సభ్యులని ఒకరితో ఒకరు పోల్చుతారు.
నీ manager నీ తరఫున **వాదించాలి** – మరియు వాదించడానికి అతనికి **సాక్ష్యం** కావాలి.
"అతను బాగా పనిచేశాడు" ఒక వాదన కాదు. "అతను X చేశాడు, ఫలితం Y" ఒక వాదన.

నీ manager కి నువ్వు ఇవ్వాల్సినవి

సంఖ్యలతో కూడిన 5-7 విజయాలు
Client / peer నుంచి వచ్చిన ప్రశంస emails
Feature కి మించి చేసినవి – POC, mentoring, docs

Bell curve అంటే

అత్యుత్తమ rating ఒక నిర్దిష్ట శాతానికే
అంటే నువ్వు బాగా చేసినా, ఇంకొకరు
ఇంకా బాగా చేస్తే – నీది తగ్గుతుంది

ఏడాది పొడవునా చేయాల్సిన ఒక్క పని

ఒక "brag document" నిర్వహించు. ఇది అహంకారం కాదు; ఇది నీ manager కి ఆయుధాలు ఇవ్వడం.

ప్రతి నెలా 10 నిమిషాలు, ఒక ఫైల్‌లో ఇలా రాయి:

```
మార్చి 2026
• Claims search module — end-to-end delivered, 0 UAT defects
• Search response time 8s → 2s (server-side pagination ప్రతిపాదించాను)
• Design system lo DataTable component రాశాను — ఇప్పుడు 4 teams వాడుతున్నాయి
• ఇద్దరు కొత్తవాళ్ళకి onboarding — KT doc రాశాను
• Client email (Mar 18) — "the team's responsiveness on the P1 was excellent"
```

Appraisal సమయంలో ఇది **అప్పటికప్పుడు గుర్తుచేసుకోవడం** కంటే వెయ్యి రెట్లు మెరుగు. మరియు నిజం ఏమిటంటే – **మార్చిలో చేసినది మార్చి తర్వాత ఎవరికీ గుర్తుండదు**, నువ్వు గుర్తుచేయకపోతే.

Gotchas (సాధారణ తప్పులు)

1. **Self-appraisal ని నిరాడంబరంగా రాయడం.** "నేను నా బాధ్యతలు నిర్వర్తించాను" – ఇది manager కి ఏమీ ఇవ్వదు. **సంఖ్యలతో, నిర్దిష్టంగా** రాయి.
2. **Manager తో సంవత్సరానికి ఒకసారి మాట్లాడటం.** నెలవారీ 1:1 లేకపోతే, calibration lo అతను నీ గురించి చెప్పడానికి ఏమీ ఉండదు.
3. **Rating ని వ్యక్తిగతంగా తీసుకోవడం.** Bell curve ఒక నిర్మాణాత్మక పరిమితి. సగటు rating అంటే "నువ్వు చెడ్డవాడివి" కాదు.
4. **Rating తర్వాత feedback అడగకపోవడం.** "తర్వాతి స్థాయికి వెళ్ళాలంటే నిర్దిష్టంగా ఏమి కావాలి?" – ఈ ప్రశ్న అడగడం, మరియు సమాధానాన్ని **రాసుకోవడం** – ఇదే వచ్చే సంవత్సరం ప్రణాళిక.

Key Points

- Rating **calibration meeting** lo నిర్ణయమవుతుంది – నీ manager అక్కడ నీ తరపున వాదిస్తాడు.
- అతనికి **సాక్ష్యం** కావాలి. **Brag document** – నెలకి 10 నిమిషాలు.
- Bell curve ఒక నిర్మాణాత్మక పరిమితి; వ్యక్తిగతంగా తీసుకోవద్దు.
- Rating తర్వాత ఎప్పుడూ అడుగు: **"తర్వాతి స్థాయికి నిర్దిష్టంగా ఏమి కావాలి?"**

Part 4 – లోపలికి రావడం (Hiring)

27. Requisition ఎక్కడ నుంచి వస్తుంది – ఇది నీ వ్యూహాన్ని మారుస్తుంది

వివరణ

Product company lo hiring ఒక దీర్ఘకాలిక ప్రణాళిక. Service company lo hiring ఒక **demand సంకేతం** – ఒక account కి మనుషులు కావాలి, ఇప్పుడు.

ఒక REQUISITION యొక్క జీవితం · ఇది నీకు ఎందుకు ముఖ్యం

CLIENT DEMAND → నీ INBOX – నాలుగు అడుగులు

1 · Client	2 · Account team	3 · Requisition	4 · నీకు call
"మాకు 8 React devs ఏప్రిల్ నుంచి కావాలి"	4 మందిని లోపల వెతుకుతుంది (bench)	మిగిలిన 4 కి బయటి నియామకం · grade fix	Recruiter / Naukri / referral

దీని నుంచి వచ్చే మూడు ఆచరణాత్మక పరిణామాలు

1 · **వేగం ముఖ్యం** – req నిండిపోతే మూసేస్తారు. వారం ఆలస్యం = అవకాశం పోయింది.
2 · **Grade ముందే నిర్ణయమైంది** – నువ్వు interview lo ఎంత బాగా చేసినా దాటలేవు (§6).
3 · **Urgency నీ మిత్రుడు** – client ఒత్తిడిలో ఉంటే rounds తగ్గుతాయి, negotiation సులభమవుతుంది.

అందుకే recruiter screen lo ఈ ప్రశ్న అడగాలి

"Is this for a specific client project, and when is the expected start date?"
"ఏప్రిల్ 1" అని సమాధానం వస్తే – వాళ్ళు తొందరలో ఉన్నారు. అది **నీ బలం**.
"General pipeline కోసం" అంటే – నెలలు పట్టొచ్చు. అంచనాలు దానికి తగ్గట్టు పెట్టుకో.

Key Points

- Requisition = ఒక **client demand** యొక్క ప్రతిధ్వని, దీర్ఘకాలిక ప్రణాళిక కాదు.
- **వేగంగా స్పందించు** – reqs నిండిపోయి మూతపడతాయి.
- Grade **req lo ముందే నిర్ణయమై** ఉంటుంది.
- "Specific project నా, general pipeline నా?" – ఇది నీ సమయాన్ని, అంచనాలని సర్దుబాటు చేసే ప్రశ్న.

---

28. Apply ఎలా – portal, Naukri, referral, vendor

వివరణ

మార్గం	స్పందన రేటు	వేగం	గమనిక
Employee referral	**అత్యధికం**	వేగం	Referral bonus ఉంది కాబట్టి ఉద్యోగులు ఆసక్తిగా ఉంటారు. **ఇదే ఉత్తమ మార్గం**
Naukri (profile updated)	ఎక్కువ	వేగం	Recruiters ఇక్కడే వెతుకుతారు. Profile ని **రోజూ update** చేస్తే search lo పైకి వస్తుంది
LinkedIn	మధ్యస్థం	మధ్యస్థం	"Open to work" ఆన్ చేయి; recruiters కి కనిపిస్తుంది
Careers portal	తక్కువ	నెమ్మది	ATS filter కఠినం; కానీ referral తో కలిపితే బలం
Vendor / consultancy	ఎక్కువ	వేగం	జాగ్రత్త: payroll vendor దా, client దా అని స్పష్టంగా అడుగు

Referral ని ఎలా అడగాలి (తెలియనివాళ్ళని కూడా)

LinkedIn lo ఆ కంపెనీలో ఉన్నవాళ్ళకి – ముఖ్యంగా **అదే role lo ఉన్నవాళ్ళకి** – ఇలా రాయి:

> "Hi Priya – I saw the Frontend Developer (React) opening for the retail account (req 1284xx). I have 3 years on React/TypeScript, currently building the claims module on an insurance platform. Would you be open to referring me? Happy to send my CV and the specific req ID so it's a two-minute job for you. No problem at all if you'd rather not."

> **ఇది ఎందుకు పనిచేస్తుంది:** నిర్దిష్టమైన req ID (నువ్వు homework చేశావని చూపిస్తుంది), సంక్షిప్తం, వాళ్ళ శ్రమని తక్కువ చేస్తుంది, మరియు **తిరస్కరించడానికి సులభమైన మార్గం ఇస్తుంది** – ఇదే దాన్ని మర్యాదగా చేస్తుంది.

Naukri profile – recruiters నిజంగా వెతికే విధానం

Recruiters keyword search చేస్తారు: `React AND TypeScript AND Redux`, experience 2-5, location. అంటే:

- **Skills section lo ప్రతి keyword ఉండాలి** – React, Redux, TypeScript, JavaScript, HTML5, CSS3, REST API, Git, Agile, Jest. లేని skill రాయొద్దు, కానీ ఉన్నదాన్ని విడిచిపెట్టొద్దు.
- **Resume headline** – "Frontend Developer | React, TypeScript, Redux | 3 Years" – సూటిగా.
- **రోజూ update చెయ్యి** (ఒక పదం మార్చినా చాలు) – Naukri search lo "recently active" profiles పైన వస్తాయి.
- **Notice period** ని నిజాయితీగా పెట్టు – ఇది filter.

Key Points

- **Referral > Naukri > LinkedIn > portal.** వీలైతే ఎప్పుడూ referral.
- Naukri profile **రోజూ update** – ఇది search ranking ని నేరుగా ప్రభావితం చేస్తుంది.

- Vendor ద్వారా అయితే – **payroll ఎవరిది** అని స్పష్టంగా అడుగు.

---

29. Recruiter screen – వాళ్ళు ఏం చూస్తున్నారు

వివరణ

మొదటి call 10–20 నిమిషాలు. Recruiter technical మనిషి కాదు – అతను ఒక **checklist** నింపుతున్నాడు:

వాళ్ళు తనిఖీ చేసేది	నీ సమాధానం ఎలా ఉండాలి
అనుభవం సరిపోతుందా	"3 years 2 months, entirely React/TypeScript"
Skills match అవుతున్నాయా	JD lo ఉన్న పదాలనే వాడు – React, Redux, REST
Notice period	నిజం చెప్పు. Buy-out ఉంటే చెప్పు
ప్రస్తుత CTC	**వీలైనంత ఆలస్యం** (§6)
Expected CTC	ఒక పరిధి చెప్పు, ఒక సంఖ్య కాదు
Location	స్పష్టంగా. Relocation సిద్ధమా కాదా
Communication	**ఇది అంచనా వేయబడుతోంది** – నువ్వు గమనించకపోయినా

నువ్వు అడగాల్సిన నాలుగు ప్రశ్నలు

ఈ నాలుగు అడగడం professional గా కనిపిస్తుంది, మరియు నీ సమయాన్ని కాపాడుతుంది:

1. **"What's the tech stack – is the backend Java or Node?"** → MERN నేపథ్యం అయితే ఇది కీలకం (§17).
2. **"What grade/level is this requisition open for?"** → §6. దీని మీదే నీ CTC పైకప్పు.
3. **"Is this for a specific client project? What's the expected start date?"** → urgency కొలత (§27).
4. **"How many rounds, and is there an online coding test?"** → తయారీ ప్రణాళిక.

> **CTC ప్రశ్నని ఎలా దాటవేయాలి (మర్యాదగా).**
>
> Recruiter: "What's your current CTC?"
>
> నువ్వు: *"I'd rather talk about the range for the role first, if that's okay – my current package includes a variable component that makes a single number misleading. Could you share the band for this requisition? I'm fairly flexible for the right role and grade."*
>
> ఇది ఎప్పుడూ పనిచేయదు – కొన్ని కంపెనీల్లో payslip అడుగుతారు. కానీ **ప్రయత్నించడం ఉచితం**, మరియు పనిచేస్తే అది నీ offer ని ₹1–2L పెంచుతుంది.

Key Points

- Recruiter ఒక **checklist** నింపుతున్నాడు – JD పదాలనే వాడు.

- నీ నాలుగు ప్రశ్నలు: **stack, grade, project/start date, rounds**.
- CTC ని ఆలస్యం చెయ్యి; expected ని **పరిధిగా** చెప్పు.
- ఈ call lo కూడా **communication అంచనా వేయబడుతోంది**.

---

30. కంపెనీల వారీగా rounds – ఒక పోలిక

వివరణ

> **ముఖ్య గమనిక:** ఈ నిర్మాణాలు **business unit ని బట్టి, project urgency ని బట్టి మారతాయి** – ఒకే కంపెనీలో ఒకరికి 2 rounds, ఇంకొకరికి 4 రావచ్చు. కాబట్టి ఇది ఒక సాధారణ ఆకారం మాత్రమే; **నీ recruiter ని నేరుగా అడగడమే** ఖచ్చితమైన సమాధానం.

కంపెనీ	సాధారణ lateral rounds	గమనికలు
Infosys	Recruiter → Technical (~1hr) → Managerial → HR	Lateral కి **aptitude/NQT లేదు**. Teams lo జరుగుతుంది; live coding ఉంటే CodeSandbox
TCS	Recruiter → Technical → Managerial/HR	కొన్ని reqs కి online assessment ఉండొచ్చు
Wipro	Recruiter → Technical → Client round → HR	**Client round** సాధారణం – client tech lead నిన్ను interview చేస్తాడు
Cognizant	Recruiter → Technical 1 → Technical 2 → HR	రెండు technical rounds సాధారణం
Accenture	Assessment (కొన్ని) → Technical → Managerial → HR	Online assessment fresher/junior reqs lo ఎక్కువ
Capgemini	Recruiter → Technical → Managerial → HR	Domain ఆధారిత ప్రశ్నలు రావొచ్చు
HCLTech / LTIMindtree	Recruiter → Technical → Managerial/HR	తరచుగా వేగవంతం

Client round – service company యొక్క ప్రత్యేకత

ఇది product companies lo ఉండదు. **Client tech lead** (అంటే నీ భవిష్యత్ project యొక్క client వైపు మనిషి) నిన్ను interview చేస్తాడు. ఇక్కడ:

- సాంకేతిక ప్రశ్నలు **సాధారణంగా తేలిక** – వాళ్ళు నీ IQ ని కొలవట్లేదు.
- వాళ్ళు చూసేది: **నువ్వు స్పష్టంగా మాట్లాడతావా, requirement ని అర్థం చేసుకుంటావా, జట్టుగా పనిచేస్తావా.**
- **Domain గురించి ఆసక్తి చూపించు** – "Claims processing lo ఈ workflow ఎలా ఉంటుంది?" అని అడిగితే అది గొప్ప సంకేతం.
- ఇక్కడ reject అయితే, కంపెనీ నిన్ను వేరే account కి పరిగణించొచ్చు – కాబట్టి ఇది ముగింపు కాదు.

Key Points

- చాలా వరకు: **Technical** → **Managerial** → **HR**. Rounds సంఖ్య BU మరియు urgency ని బట్టి **మారుతుంది**.
- Lateral track కి **aptitude test సాధారణంగా ఉండదు** (అది fresher pipeline).
- **Client round** (Wipro, కొన్ని accounts) – సాంకేతికం కంటే **సరిపోలిక** పరీక్ష.
- ఖచ్చితమైన నిర్మాణం కోసం **recruiter ని అడుగు** – ఇది ఊహించే విషయం కాదు.

---

31. Technical round – 2-5 years కి నిజంగా ఏం అడుగుతారు

వివరణ

ఇది ఒక గంట. అందులో సుమారు ఈ విభజన:

ఒక గంట TECHNICAL ROUND · సమయం ఎక్కడికి పోతుంది

SERVICE MNC · FRONTEND · 2-5 సంవత్సరాలు

పరిచయం	~5 నిమి – "Tell me about yourself" + project పరిచయం
JavaScript fundamentals	~15 నిమి – **ఇక్కడే ఎక్కువ మంది reject అవుతారు**
React	~15 నిమి – hooks, state, re-render, performance
Project లోతు	~10 నిమి – "నువ్వు ఏం చేశావు?" (§37 తో కలుస్తుంది)
Coding	~10 నిమి – చిన్న JS/React task (§35)
నీ ప్రశ్నలు	~5 నిమి – ఇది కూడా అంచనా వేయబడుతుంది

అతి ముఖ్యమైన గమనిక: రెండు అతిపెద్ద భాగాలు **JavaScript** మరియు **React** – కానీ reject కి అత్యంత సాధారణ కారణం JSX కాదు, **vanilla JavaScript**.

Candidates `useEffect` నియమాలని అప్పజెప్పగలరు కానీ closures, event loop, prototype ప్రశ్నల దగ్గర పడిపోతారు. అందుకే prep lo **~40% vanilla JavaScript** కి కేటాయించడం సరైన విభజన – React patterns కి కాదు.

Difficulty స్థాయి – నిజాయితీగా

Service MNC technical rounds ని candidates స్థిరంగా **"easy to medium"** అని వర్ణిస్తారు. ఇది ఒక ఉపశమనం, కానీ ఒక ఉచ్చు కూడా:

- **ఉపశమనం:** నీకు LeetCode Hard అవసరం లేదు. Dynamic programming, graph algorithms – దాదాపు రావు.
- **ఉచ్చు:** "సులభం" అంటే **ప్రతి ఒక్కరూ** సమాధానం చెప్పగలరు. అంటే నువ్వు వేరుగా కనిపించాలంటే – సమాధానం కాదు, **సమాధానం చెప్పే విధానం** ముఖ్యం.

> ఉదాహరణ: "What is a closure?" అనే ప్రశ్నకి అందరూ నిర్వచనం చెప్తారు. నువ్వు నిర్వచనం + **ఒక నిజమైన bug** చెప్పు: *"...and the place this actually bit us was a loop that attached click handlers – all of them logged the last index because they closed over the same variable. We fixed it with* `let` *instead of* `var`*."* – ఇప్పుడు నువ్వు వేరు.

Key Points

- విభజన: JS ~15 నిమి, React ~15 నిమి, project ~10, coding ~10.
- Difficulty **easy-medium** – కానీ అందుకే **విధానం** ముఖ్యం, సమాధానం కాదు.
- Reject కి #1 కారణం: **vanilla JavaScript**, React కాదు.
- ప్రతి భావనకి **ఒక నిజమైన అనుభవం** జత చెయ్యి.

---

32. JavaScript bank – నిజమైన eliminator

వివరణ

ఇది ప్రశ్నల జాబితా మాత్రమే కాదు – **ప్రతిదానికీ ఎందుకు అడుగుతున్నారో** ఉంది. లోతైన వివరణలు `JavaScript_Telugu.md` lo ఉన్నాయి; ఇక్కడ ఉన్నది **interview కోణం**.

దాదాపు ఖచ్చితంగా వచ్చేవి (ప్రతిసారీ సిద్ధంగా ఉండు)

ప్రశ్న	వాళ్ళు నిజంగా పరీక్షిస్తున్నది
`var` vs `let` vs `const`	Scope మరియు TDZ అర్థమయ్యిందా
Closure అంటే ఏమిటి	నువ్వు నిర్వచనం అప్పజెప్తున్నావా, అనుభవంతో మాట్లాడుతున్నావా
Event loop – `setTimeout(...,0)` ఎప్పుడు నడుస్తుంది	Async mental model ఉందా
`this` **binding** – ఎవరు నిర్ణయిస్తారు	Arrow vs regular function తేడా
Promise vs async/await, `Promise.all` vs `allSettled`	నిజమైన async code రాశావా
Prototype chain / inheritance	భాష ఎలా పనిచేస్తుందో తెలుసా
Shallow vs deep copy	Spread ఎక్కడ ఆగుతుందో తెలుసా
`map` **/** `filter` **/** `reduce`	రోజువారీ సాధనాలు
Debounce vs throttle – తేడా మరియు ఎప్పుడు ఏది	Frontend-నిర్దిష్ట ఆలోచన
Hoisting	Function vs variable declarations

తరచుగా వచ్చేవి

- `==` vs `===`, మరియు type coercion యొక్క ఆశ్చర్యాలు
- `null` vs `undefined`
- Event bubbling, capturing, delegation
- `localStorage` vs `sessionStorage` vs cookies (మరియు security పరిణామాలు)
- CORS అంటే ఏమిటి, ఎవరు దాన్ని అమలు చేస్తారు (**browser**, server కాదు)
- Optional chaining `?.`, nullish coalescing `??`
- Module systems – ESM vs CommonJS

- `Array.prototype` methods – ఏవి mutate చేస్తాయి, ఏవి చేయవు
- Currying, partial application (అరుదు కానీ వస్తుంది)

ఒక ప్రశ్నకి "సరైన" సమాధాన నిర్మాణం

> **ప్రశ్న:** "What is the difference between debounce and throttle?"
>
> **సాధారణ సమాధానం:** "Debounce waits until the user stops; throttle runs at fixed intervals." *(సరైనది – కానీ ప్రతి ఒక్కరూ ఇదే చెప్తారు.)*
>
> **వేరుగా నిలిచే సమాధానం:** "Debounce waits for a pause before firing; throttle fires at most once per interval. In our app we used debounce on the claims search input at 300ms, so we weren't firing a request per keystroke – that alone cut search traffic by about 80%. We used throttle on a scroll handler that repositioned a sticky header, because there you want steady updates rather than one at the end. The rule I use is: debounce when only the final state matters, throttle when the intermediate states matter too."

> **నిర్మాణం:** నిర్వచనం (1 వాక్యం) → **నిజమైన ఉదాహరణ + సంఖ్య** → **ఎప్పుడు ఏది అనే నియమం**. ఈ మూడు అడుగుల నమూనాని ప్రతి సమాధానానికి వాడు.

Gotchas

1. **నిర్వచనం అప్పజెప్పి ఆగిపోవడం.** ఇది "చదివాడు" అని చెప్తుంది, "వాడాడు" అని కాదు.
2. **"నాకు తెలియదు" అని చెప్పకుండా ఊహించడం.** మంచి సమాధానం: *"I haven't used Proxy in production, but my understanding is it intercepts property access – is that where you're going?"* – నిజాయితీ + ఆసక్తి.
3. **Vanilla JS ని విస్మరించి React మీద మాత్రమే prep చేయడం.** ఇదే #1 reject కారణం.

Key Points

- Prep సమయంలో **~40% vanilla JavaScript** – ఇదే eliminator.
- ప్రతి సమాధానం: **నిర్వచనం → నిజమైన ఉదాహరణ + సంఖ్య → నియమం**.
- తెలియకపోతే **నిజాయితీగా చెప్పు** + నీ అంచనా చెప్పు.

---

33. React bank

వివరణ

దాదాపు ఖచ్చితంగా వచ్చేవి

ప్రశ్న	వాళ్ళు చూస్తున్నది
`useState` **vs** `useReducer` – ఎప్పుడు ఏది	సంక్లిష్టత ఎప్పుడు మారుతుందో తెలుసా
`useEffect` **dependency array** – ఖాళీ vs లేకపోవడం vs విలువలు	అత్యంత సాధారణ bug మూలం

ప్రశ్న	వాళ్ళు చూస్తున్నది
Cleanup function ఎందుకు కావాలి	Memory leaks, race conditions
Component ఎప్పుడు re-render అవుతుంది	4 కారణాలు: own state, parent render, context, key మార్పు
`useMemo` / `useCallback` – ఎప్పుడు వాడాలి, ఎప్పుడు వద్దు	Premature optimization గుర్తిస్తావా
`React.memo` ఎందుకు పనిచేయదు కొన్నిసార్లు	Props identity అర్థమయ్యిందా
Keys in lists – index ఎందుకు చెడ్డది	Reconciliation అర్థమయ్యిందా
Controlled vs uncontrolled components	Forms ఎలా నిర్వహిస్తావు
Context API – ఎప్పుడు సరిపోతుంది, ఎప్పుడు కాదు	State management ఆలోచన
Custom hook ఒకటి రాసి చూపించు	ఆచరణాత్మక React

తరచుగా వచ్చేవి

- Redux Toolkit – slice, thunk, RTK Query ఎందుకు
- Server state vs client state (React Query / RTK Query ఎందుకు వచ్చాయి)
- Error boundary – ఏమి పట్టుకుంటుంది, ఏమి పట్టుకోదు (event handlers, async కాదు)
- Lazy loading, `React.lazy` + `Suspense`, route-based code splitting
- Virtualization (`react-window`) పెద్ద lists కి
- `useRef` – DOM access మరియు render కాకుండా విలువ నిల్వ
- Prop drilling vs composition (`children` pass చేయడం)
- Strict mode double-render dev lo ఎందుకు
- Testing – RTL తో component ఎలా test చేస్తావు

"Component ఎప్పుడు re-render అవుతుంది" – ఒక ఆదర్శ సమాధానం

"Four things trigger a re-render: its own state changes, its parent re-renders, a context it consumes changes, or its key changes – which is a remount, not a re-render. The one that surprises people is the second: a parent re-rendering re-renders every child by default, even if the props are identical. `React.memo` stops that, but only if the props are referentially stable – so passing an inline arrow or an object literal defeats it, which is why `useCallback` and `useMemo` exist. In practice I reach for composition first: moving state down, or passing children through, usually removes the problem without any memoisation at all."

> ఇందులో ఏముంది: పూర్తి జాబితా, ఒక **ఆశ్చర్యకరమైన విషయం**, memo ఎందుకు విఫలమవుతుంది, మరియు **memoisation కంటే composition మెరుగు** అనే పరిణతమైన అభిప్రాయం.

Gotchas

1. `useMemo` **ని ప్రతిచోటా సూచించడం.** Interviewers దీన్ని ఒక cargo-cult సంకేతంగా చూస్తారు. "మొదట measure చేస్తాను" అనేది మంచి సమాధానం.
2. **Class components గురించి పూర్తిగా మర్చిపోవడం.** Legacy codebases lo ఇవి ఉంటాయి (§13). `componentDidMount` → `useEffect` mapping తెలియాలి.
3. **"Redux వాడాను" అని చెప్పి ఎందుకో చెప్పలేకపోవడం.** *"Global state కావాలి కాబట్టి" సరిపోదు. "ఐదు screens ఒకే user permissions ని చదువుతాయి, మరియు మాకు time-travel debugging కావాలి"* – ఇది సమాధానం.

Key Points

- Re-render యొక్క **4 కారణాలు** – ఇది దాదాపు ఖచ్చితంగా వస్తుంది.
- `useMemo` / `useCallback` గురించి **పరిణతి** చూపించు – "measure చేసి, composition ముందు".
- ప్రతి library ఎంపికకి **ఒక కారణం** సిద్ధంగా ఉంచు.
- లోతు కోసం `React_Telugu.md` – ఇక్కడ ఉన్నది interview కోణం మాత్రమే.

---

34. HTML / CSS / Accessibility bank

వివరణ

ఈ విభాగాన్ని చాలా మంది నిర్లక్ష్యం చేస్తారు – మరియు అదే ఒక అవకాశం. Enterprise projects lo CSS మరియు a11y నిజమైన సమస్యలు (§16).

CSS – వచ్చే ప్రశ్నలు

ప్రశ్న	గమనిక
Flexbox vs Grid – ఎప్పుడు ఏది	Flex = ఒక అక్షం; Grid = రెండు అక్షాలు
Box model, `box-sizing: border-box`	ప్రాథమికం కానీ తరచుగా అడుగుతారు
Position – relative, absolute, fixed, sticky	`sticky` ఎందుకు పనిచేయదు (parent overflow)
Specificity ఎలా లెక్కిస్తారు	`!important` ఎందుకు చివరి ఆశ్రయం
Responsive – media queries, mobile-first, `clamp()`	Enterprise lo breakpoints client నిర్ణయిస్తాడు
CSS Modules vs styled-components vs Tailwind	ఎందుకు ఏది ఎంచుకున్నావు
Stacking context / `z-index` పనిచేయకపోవడం	Modal bugs కి సాధారణ కారణం
Reflow vs repaint	Performance ప్రశ్నలకి దారి

HTML & semantics

- Semantic elements – `<header>`, `<nav>`, `<main>`, `<article>`, `<button>` vs `<div>`
- Form elements మరియు `<label>` association
- `<img>` alt text – ఎప్పుడు ఖాళీగా ఉంచాలి
- Data attributes, `aria-*` attributes మధ్య తేడా

Accessibility – ఇక్కడ నువ్వు మెరవగలవు

§16 lo ఉన్న జాబితా చదువు. Interview lo అడిగే విధానం సాధారణంగా ఇది:

> **"How do you make a modal accessible?"**

"Four things. First, focus has to move into the dialog when it opens and return to the trigger when it closes – otherwise a keyboard user is dropped at the top of the page. Second, focus has to be trapped inside while it's open, so Tab cycles within it. Third, Escape should close it, and clicking the backdrop should too, but only if there's no unsaved input. Fourth, the container needs `role="dialog"` with `aria-modal="true"` and an `aria-labelledby` pointing at the title, and the background should be inert so a screen reader doesn't read through it. We put all of that inside the design-system Modal, so no feature team has to remember it."

> చివరి వాక్యం మళ్ళీ **systems ఆలోచన** – ఇదే senior సంకేతం.

Key Points

- CSS/a11y ని చాలా మంది నిర్లక్ష్యం చేస్తారు – అందుకే ఇది **వేరుపాటు అవకాశం**.
- `z-index` పనిచేయకపోవడం = **stacking context** – ఇది ఒక క్లాసిక్ ప్రశ్న.
- Modal accessibility: **focus move, focus trap, Escape, ARIA + inert background**.
- ఎప్పుడూ "మేము దీన్ని design system lo పెట్టాం" అని ముగించు.

---

35. చిన్న coding task – machine coding round

వివరణ

ముందుగా ఒక అంచనా సర్దుబాటు. పూర్తి స్థాయి **90-నిమిషాల machine coding round** అనేది ఎక్కువగా **product companies మరియు GCCs** నమూనా. Service MNCs (TCS, Infosys, Wipro, Cognizant, Accenture) సాధారణంగా **సంభాషణాత్మక technical round** lo ఒక **10–20 నిమిషాల చిన్న task** ఇస్తాయి. కాబట్టి – **recruiter ని format అడిగి** దానికి తగ్గట్టు prep చెయ్యి, ఊహించొద్దు.

సాధారణంగా ఇచ్చే చిన్న tasks

Task	పరీక్షిస్తున్నది
Counter / Todo list	ప్రాథమిక state, list rendering, keys
Search with debounce	Async, cleanup, closures – **అత్యంత సాధారణం**
Accordion / Tabs	Conditional rendering, a11y (bonus)
Modal	Portal, focus, Escape handling
Star rating / toggle group	Controlled components, event handling

Task	పరీక్షిస్తున్నది
Fetch + list + loading/error	నిజమైన data flow – **ఇది ఎక్కువగా వస్తుంది**
Pagination / infinite scroll	IntersectionObserver లేదా scroll handler
Autocomplete	పైవన్నీ కలిపి – ఇది "పెద్ద" version

ఒక చిన్న task ని ఎలా చేయాలి – ఒక పద్ధతి

1. మొదట ప్రశ్నలు అడుగు (30 సెకన్లు). ఇది సమయం వృథా కాదు – ఇదే మొదటి points.

> *"Quick clarifications: should the search hit an API or filter a local array? Do you want me to handle loading and error states? And should I worry about accessibility, or focus on the core logic first?"*

2. బిగ్గరగా ఆలోచించు. మౌనంగా type చేయడం అత్యంత సాధారణ తప్పు. Interviewer నీ ఆలోచనని అంచనా వేస్తున్నాడు, నీ typing speed ని కాదు.

3. పనిచేసేది ముందు, అందమైనది తర్వాత. మొదట పనిచేసే version. తర్వాత *"Now let me clean this up – I'd extract this into a `useDebounce` hook."*

4. Edge cases ని బిగ్గరగా చెప్పు – అమలు చేయకపోయినా.

> *"I haven't handled the race condition where an older request resolves after a newer one – in production I'd use an AbortController or track the latest request id. Should I add it?"*
>
> **ఇది చెప్పడం, అమలు చేయడం అంత విలువైనది** – మరియు ఇది కేవలం 8 సెకన్లు.

5. చివర్లో సొంతంగా విమర్శించు. *"If I had more time I'd add tests for the debounce timing and move the fetch into a custom hook."*

ఒక నమూనా – debounced search

```
// ఇది interview lo రాయగలిగే స్థాయి – పూర్తి, కానీ అతిగా కాదు
function useDebounced(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);   // ← cleanup: ఇది లేకపోతే ప్రతి keystroke కి timer
  }, [value, delay]);
  return debounced;
}

function Search() {
  const [query, setQuery] = useState('');
  const [items, setItems] = useState([]);
  const [status, setStatus] = useState('idle');   // idle | loading | error | done
  const debounced = useDebounced(query);
```

```
  useEffect(() => {
    if (!debounced) { setItems([]); setStatus('idle'); return; }
    const ctrl = new AbortController();            // ← race condition రక్షణ
    setStatus('loading');
    fetch(`/api/search?q=${encodeURIComponent(debounced)}`, { signal: ctrl.signal })
      .then((r) => { if (!r.ok) throw new Error(r.status); return r.json(); })
      .then((data) => { setItems(data); setStatus('done'); })
      .catch((e) => { if (e.name !== 'AbortError') setStatus('error'); });
    return () => ctrl.abort();                     // ← cleanup
  }, [debounced]);

  return (
    <div>
      <label htmlFor="q">Search claims</label>
      <input id="q" value={query} onChange={(e) => setQuery(e.target.value)} />
      {status === 'loading' && <p role="status">Searching…</p>}
      {status === 'error' && <p role="alert">Something went wrong. Try again.</p>}
      {status === 'done' && items.length === 0 && <p>No results for “{debounced}”.
</p>}
      <ul>{items.map((it) => <li key={it.id}>{it.title}</li>)}</ul>
    </div>
  );
}
```

> ఈ 30 lines lo ఉన్నవి: debounce, cleanup, **AbortController**, నాలుగు states (idle/loading/error/empty), `label` association, `role="status"` / `role="alert"`, మరియు సరైన `key`. **ఇది ఒక 2-సంవత్సరాల candidate కి, ఒక 4-సంవత్సరాల candidate కి మధ్య తేడా.**

Key Points

- Service MNC lo సాధారణంగా **చిన్న task**, పూర్తి machine coding round కాదు – **format ని అడుగు**.
- పద్ధతి: **ప్రశ్నలు → బిగ్గరగా ఆలోచించు → పనిచేసేది ముందు → edge cases చెప్పు → సొంత విమర్శ**.
- Loading/error/empty states – ఇవి లేకపోతే solution అసంపూర్ణం.
- **AbortController ప్రస్తావించడం** ఒక బలమైన సంకేతం.

---

36. Frontend system design – SSE స్థాయిలో

వివరణ

4+ సంవత్సరాల reqs lo ఒక తేలికపాటి design ప్రశ్న వస్తుంది. ఇది distributed systems కాదు – ఇది **frontend architecture**.

సాధారణ ప్రశ్నలు: *"Design a dashboard with 20 widgets"*, *"How would you structure a large React app?"*, *"Design an autocomplete for 10 million products"*, *"How do you handle a table with 100,000 rows?"*

ఒక చట్రం – ఏ ప్రశ్నకైనా వాడొచ్చు

FRONTEND SYSTEM DESIGN · ఆరు అడుగుల చట్రం

ఏ frontend design ప్రశ్నకైనా ఇదే క్రమం

1 · Requirements స్పష్టం చెయ్యి	ఎంతమంది users? ఏ devices? Real-time నా?
2 · Component విభజన	ఏవి స్వతంత్రం, ఏవి ఆధారపడతాయి
3 · Data flow & state	Server state vs client state; ఎక్కడ ఉంచాలి
4 · API contract	ఎన్ని calls? Batch నా? Pagination ఎలా?
5 · Performance	Code splitting, virtualization, caching, memo
6 · Edge cases	Error, empty, loading, offline, a11y, i18n

చాలా candidates 2 మరియు 5 మాత్రమే మాట్లాడతారు. **1, 4, 6 మాట్లాడేవాడు వేరుగా నిలుస్తాడు.**

"1,00,000 rows ఉన్న table" – ఒక నమూనా సమాధానం

"First, a clarifying question – does the user actually need all hundred thousand rows on screen, or do they need to find something in them? That changes the answer completely.

If it's find-something, the right design is server-side search and pagination, and the UI never holds more than a page. That's the cheapest option and usually the correct one.

If they genuinely need to scan the whole set – say a reconciliation screen – then I'd virtualise with something like react-window, so we render maybe thirty rows regardless of the data size. Sorting and filtering move to the server, because sorting a hundred thousand rows in JavaScript blocks the main thread for seconds.

The parts people miss: keyboard navigation and screen-reader semantics break under virtualisation because the rows aren't in the DOM, so we'd need proper `aria-rowcount` and managed focus. And I'd check the actual row height – variable-height rows make virtualisation substantially harder, so if the design allows fixed heights, that's worth agreeing early with the designer."

> **ఇందులో senior సంకేతాలు:** requirement ని ప్రశ్నించడం, **సులభమైన పరిష్కారాన్ని ముందు ప్రతిపాదించడం**, trade-off, a11y ప్రభావం, మరియు designer తో ముందే మాట్లాడాలనే ఆచరణాత్మక సలహా.

Key Points

- Frontend design = **components + state + API + performance + edge cases**, distributed systems కాదు.
- ఎప్పుడూ **requirement ని ప్రశ్నించడంతో** మొదలుపెట్టు.
- **సులభమైన పరిష్కారాన్ని ముందు ప్రతిపాదించు** – virtualization కంటే pagination.

- **a11y మరియు i18n ప్రస్తావించడం** – ఇది చాలా అరుదు, కాబట్టి చాలా బలమైనది.

---

37. Managerial round – project ownership కథ

వివరణ

ఇక్కడే ఎక్కువమంది experienced candidates reject అవుతారు – technical round lo కాదు. Service company కి కావలసింది client ముందు కూర్చోబెట్టగలిగే మనిషి; ఈ round అదే పరీక్షిస్తుంది.

STAR – కానీ సరిగ్గా వాడితే

Situation → **T**ask → **A**ction → **R**esult. చాలా మందికి ఇది తెలుసు, కానీ ఇక్కడ రెండు తప్పులు జరుగుతాయి:

1. Situation కి 2 నిమిషాలు ఇచ్చి, Action కి 20 సెకన్లు ఇవ్వడం. **సరైన నిష్పత్తి: S+T = 25%, A = 50%, R = 25%.**
2. **R (Result) lo సంఖ్య లేకపోవడం.** "బాగా జరిగింది" ఒక ఫలితం కాదు.

సిద్ధంగా ఉంచుకోవాల్సిన ఆరు కథలు

ఇవి ఆరు – ప్రతి managerial round ప్రశ్న వీటిలో ఒకదానికి వస్తుంది:

కథ	అడిగే ప్రశ్న
1. నీవు own చేసిన feature	"Tell me about something you delivered end-to-end"
2. నీవు పరిష్కరించిన కష్టమైన bug	"Describe a technical challenge"
3. నీకు, ఇంకొకరికి మధ్య విభేదం	"Tell me about a conflict with a teammate"
4. Deadline తప్పిన / ఒత్తిడిలో ఉన్న సందర్భం	"How do you handle tight deadlines?"
5. నీవు చేసిన తప్పు	"Tell me about a failure"
6. నీవు ఇంకొకరికి సహాయం చేసిన సందర్భం	"Have you mentored anyone?"

కథ 3 (విభేదం) – ఒక మంచి నమూనా:

"On the claims module, our backend lead wanted to return the full claim object on the list endpoint, and I wanted a trimmed version. It sounds small but the payload was about 40KB per row, and the list showed fifty.

I didn't push it in the group call. I measured it first – the list took 4.2 seconds on a throttled connection, and 90% of that was payload we never rendered. Then I took the numbers to him one-on-one and asked what the cost would be on his side to add a summary endpoint. It turned out to be about half a day, mostly because he'd been worried about maintaining two shapes.

We agreed on a summary endpoint for the list and the full object on detail. The list went to 800 milliseconds. What I'd do differently: I should have raised it during grooming rather than after he'd already built it – that half-day was avoidable."

> **ఇందులో ఏముంది:** సాక్ష్యం (4.2s, 40KB), **బహిరంగంగా ఘర్షణ పడలేదు**, అవతలివాళ్ళ ఆందోళనని అర్థం చేసుకున్నాడు, ఫలితం ఒక సంఖ్యతో, మరియు చివరిలో **సొంత తప్పుని ఒప్పుకోవడం**. ఆ చివరి వాక్యమే ఈ సమాధానాన్ని "మంచిది" నుంచి "అద్భుతం" కి తీసుకెళ్తుంది.

"నీ బలహీనత ఏమిటి" – నిజాయితీగా

Template సమాధానాలు ("నేను చాలా perfectionist") అందరికీ తెలుసు, మరియు అవి చెడ్డ సంకేతం. మంచి సమాధాన నిర్మాణం: **నిజమైన బలహీనత + దానితో ఏమి జరిగింది + ఇప్పుడు ఏమి చేస్తున్నావు.**

> *"I used to under-communicate when I was stuck – I'd try to solve it alone for two days before asking. On the payments integration that cost us most of a sprint. Now I have a hard rule: if I'm blocked for more than half a day, it goes in the standup and I message the person directly. It felt like admitting failure at first, but it's actually made me look more reliable, not less."*

Key Points

- **ఇక్కడే ఎక్కువ rejections** – technical round కంటే ఎక్కువ.
- STAR నిష్పత్తి: **S+T 25%, A 50%, R 25%**; R lo ఎప్పుడూ ఒక **సంఖ్య**.
- **ఆరు కథలు** ముందే సిద్ధం చేసుకో.
- విభేదం కథలో: సాక్ష్యం + వ్యక్తిగతంగా మాట్లాడటం + **సొంత తప్పు ఒప్పుకోవడం**.

---

38. HR round – negotiation, notice period, documents

వివరణ

HR round సాధారణంగా చిన్నది, కానీ ఇక్కడ **డబ్బు నిర్ణయమవుతుంది**. కొన్ని ఆచరణాత్మక నియమాలు:

Negotiation – ఐదు నియమాలు

1. **మొదటి సంఖ్యని నువ్వు చెప్పకు, వీలైతే.** "What are your expectations?" → *"I'd like to understand the band for this grade first – what range are you working with?"*
2. **చెప్పాల్సి వస్తే, పరిధి చెప్పు, పైవైపు మొగ్గుతూ.** "₹15–17L" అంటే వాళ్ళు ₹15L వింటారు. కాబట్టి నీ లక్ష్యం ₹16L అయితే "₹17–19L" అను.
3. **Fixed pay గురించి మాట్లాడు, CTC గురించి కాదు** (§10).
4. **ఇంకో offer ఉంటే చెప్పు – కానీ నిజాయితీగా.** అబద్ధమైన offer చెప్పడం ప్రమాదకరం; చాలా కంపెనీలు proof అడుగుతాయి.
5. **నిశ్శబ్దం ఒక సాధనం.** వాళ్ళు సంఖ్య చెప్పాక, వెంటనే "సరే" అనొద్దు. *"Let me think about that – can I come back to you tomorrow?"* – ఈ ఒక్క వాక్యం తరచుగా ₹50,000–1,00,000 తెస్తుంది.

ఒక పూర్తి negotiation మలుపు:

HR: "We can offer 14.5 lakhs total CTC."

నువ్వు: *"Thank you – I'm genuinely interested in the role. Could you break that down into fixed and variable for me? ... Okay, so 10.2 fixed. To be straightforward with you, I have another process at a similar stage where the fixed component is around 12. I'd prefer to join here because of the account and the React work specifically – is there room to move the fixed closer to that?"*

ఇది ఎందుకు పనిచేస్తుంది: కృతజ్ఞత, నిర్దిష్టత (fixed, CTC కాదు), నిజమైన ప్రత్యామ్నాయం, మరియు – ముఖ్యంగా – **నువ్వు ఇక్కడే చేరాలనుకుంటున్నావని స్పష్టంగా చెప్పడం**. Negotiation అంటే బెదిరించడం కాదు.

Notice period – ముందే ప్రణాళిక

- Service MNCs lo **60-90 రోజులు** సాధారణం.
- **Buy-out** ఉందా అని అడుగు – కొత్త కంపెనీ చెల్లిస్తుందా, లేదా నువ్వేనా.
- Offer letter lo **joining date** ఉంటుంది. అది వాస్తవికంగా ఉందా చూడు – notice extend అయితే offer రద్దు కావొచ్చు.

Documents – ముందే సిద్ధం చేసుకో

BGV (background verification) ఆలస్యం అయితే joining ఆలస్యమవుతుంది. ఇవి సిద్ధంగా ఉంచు:

- గత 3-6 నెలల **payslips**
- **Offer letter + relieving letter** గత కంపెనీల నుంచి (relieving letter లేకపోతే ఇది పెద్ద సమస్య)
- **Form 16** / ITR
- విద్యా certificates (10th నుంచి degree వరకు)
- PAN, Aadhaar, address proof
- **Gap ఉంటే** ఒక వివరణ సిద్ధం చేసుకో – నిజాయితీగా

Gotchas

1. **Verbal హామీలని నమ్మడం.** "6 నెలల్లో promotion ఇస్తాం", "onsite పంపిస్తాం" – **offer letter lo లేకపోతే అది లేదు.**

2. **Resign చేసి offer letter కోసం ఎదురుచూడటం. ఎప్పుడూ** offer letter చేతికి వచ్చాకే resign చెయ్యి.
3. **Relieving letter ని తక్కువ అంచనా వెయ్యడం.** భారతీయ IT lo ఇది కీలకం. దానికోసం notice period ని సరిగ్గా serve చెయ్యి.
4. **Joining bonus clawback ని చదవకపోవడం** (§10).

Key Points

- మొదటి సంఖ్య చెప్పకు; చెప్పాల్సి వస్తే **పరిధి, పైవైపు మొగ్గుతూ**.
- **Fixed pay** గురించి మాట్లాడు, CTC గురించి కాదు.
- *"Let me think about it"* – ఒక వాక్యం, తరచుగా వేల రూపాయలు.
- **Offer letter చేతికి వచ్చాకే resign.** Verbal హామీలు లెక్కలోకి రావు.
- BGV documents ముందే సిద్ధం – ముఖ్యంగా **relieving letter**.

Part 5 – లోపలికి వచ్చాక ఎదగడం

39. మొదటి 90 రోజులు – కొత్త account lo

వివరణ

మొదటి మూడు నెలల్లో నీ మీద ఏర్పడే అభిప్రాయం **తర్వాతి రెండేళ్ళు నిలుస్తుంది**. ఇది అన్యాయం, కానీ నిజం. ఒక స్పష్టమైన ప్రణాళిక:

మొదటి 90 రోజులు · ఏ దశలో ఏది

కొత్త account · మూడు దశలు

రోజులు 1-30 · నేర్చుకో
Codebase చదువు – feature కాదు, ప్రవాహం
ప్రతి ఒక్కరి పేరు, పాత్ర తెలుసుకో
Environment పూర్తిగా setup చెయ్యి
చిన్న tickets తీసుకో – వేగంగా ముగించు
లక్ష్యం: మొదటి PR వారం 1 lo

రోజులు 31-60 · తోడ్పడు
పూర్తి feature ownership తీసుకో
Grooming lo ప్రశ్నలు అడగడం మొదలుపెట్టు
ఇతరుల PRs review చెయ్యి
ఒక చిన్న నొప్పిని సొంతంగా పరిష్కరించు
లక్ష్యం: "ఇతన్ని అడగొచ్చు" అనే స్థితి

రోజులు 61-90 · స్థిరపడు
ఒక ప్రాంతానికి go-to వ్యక్తి అవ్వు
Client call lo మాట్లాడు
Documentation lo ఖాళీని పూరించు
Manager తో మొదటి 1:1 feedback
లక్ష్యం: దృశ్యత + నమ్మకం

మొదటి 30 రోజుల్లో అత్యంత విలువైన ఒక్క పని
నువ్వు onboarding lo పడ్డ ప్రతి ఇబ్బందిని రాసుకో – setup steps, తప్పిపోయిన access, గందరగోళ పేర్లు.
వాటిని ఒక **onboarding doc** గా మార్చు. ఇది 2 గంటల పని, మరియు తర్వాతి ప్రతి కొత్తవాడికీ ఉపయోగం.

మొదటి 90 రోజుల్లో చేయకూడనివి
"మేము ఇలా చేయాలి" అని పెద్ద మార్పులు ప్రతిపాదించడం · పాత code ని బహిరంగంగా విమర్శించడం · మౌనంగా ఇబ్బంది పడటం · ప్రశ్నలు అడగడానికి సిగ్గుపడటం (మొదటి నెలలో ప్రశ్నలు ఉచితం)

Codebase ని ఎలా చదవాలి (feature by feature కాదు)

కొత్త repo lo "అన్ని files చదవడం" పనిచేయదు. బదులుగా **ఒక ప్రవాహాన్ని** అనుసరించు:

1. ఒక screen ఎంచుకో (ఉదా. login).
2. Route ఎక్కడ నిర్వచించారో కనుక్కో.
3. ఆ component ని కనుక్కో.
4. అది ఏ API call చేస్తుందో కనుక్కో.
5. ఆ response state lo ఎక్కడ వెళ్తుందో కనుక్కో.

6. అది తిరిగి UI lo ఎక్కడ కనిపిస్తుందో కనుక్కో.

> ఈ ఒక్క ప్రవాహం పూర్తిగా అర్థమైతే – **మిగతా 90% codebase అదే నమూనాని అనుసరిస్తుంది**.
> ఇది 3 రోజుల పని, మరియు అన్ని files చదవడం కంటే వంద రెట్లు ప్రభావవంతం.

Key Points

- **1-30 నేర్చుకో, 31-60 తోడ్పడు, 61-90 స్థిరపడు.**
- **మొదటి PR వారం 1 lo** – చిన్నదైనా పర్వాలేదు; ఇది pipeline పనిచేస్తుందని రుజువు.
- Codebase ని **ఒక ప్రవాహం** గా చదువు, files జాబితాగా కాదు.
- **Onboarding doc రాయడం** – మొదటి నెలలో అత్యధిక ప్రతిఫలం ఉన్న పని.
- మొదటి 90 రోజుల్లో పెద్ద మార్పులు ప్రతిపాదించొద్దు.

---

40. Rating తెచ్చేది vs promotion తెచ్చేది – ఇవి వేరు

వివరణ

ఇది చాలా మంది గమనించని ఒక సూక్ష్మమైన, కానీ నిర్ణయాత్మకమైన తేడా:

రెండు వేరే ప్రశ్నలు · రెండు వేరే సమాధానాలు

"మంచి rating" మరియు "promotion" ఒకటి కాదు

RATING తెచ్చేది	PROMOTION తెచ్చేది
ప్రశ్న: "ఇతను తన పాత్రని ఎంత బాగా చేశాడు?"	ప్రశ్న: "ఇతను ఇప్పటికే తర్వాతి పాత్ర చేస్తున్నాడా?"
Sprint commitments నిలబెట్టుకోవడం	ఇతరులకి unblock చేయడం, mentoring
తక్కువ UAT defects	Design నిర్ణయాలు తీసుకోవడం
సమయానికి delivery	Client ముందు స్వతంత్రంగా నిలబడటం
నమ్మదగినతనం, communication	తన ticket కి మించి బాధ్యత తీసుకోవడం
= నీ ప్రస్తుత స్థాయిలో శ్రేష్ఠత	**= తర్వాతి స్థాయి ప్రవర్తన**

ఇక్కడే ఉచ్చు ఉంది

నువ్వు ఐదు సంవత్సరాలు అద్భుతమైన rating తెచ్చుకోవచ్చు – మరియు promote కాకపోవచ్చు.
ఎందుకంటే rating "నీ పాత్రలో నువ్వు మంచివాడివి" అని చెప్తుంది; promotion "నువ్వు **ఇప్పటికే** తర్వాతి పాత్ర చేస్తున్నావు" అని చెప్తుంది. Promotion ఒక బహుమతి కాదు – అది ఇప్పటికే జరుగుతున్నదాన్ని **గుర్తించడం**.

ఆచరణలో ఏమి చేయాలి

1. **తర్వాతి స్థాయి ప్రవర్తనని ముందే మొదలుపెట్టు** – అనుమతి కోసం ఎదురుచూడకుండా. Junior కి సహాయం చెయ్యి, design doc రాయి, client call lo ఒక భాగం తీసుకో.
2. **నీ manager కి స్పష్టంగా చెప్పు.** *"నేను తర్వాతి grade వైపు పనిచేస్తున్నాను. దానికి ఇంకా ఏమి కావాలి?"* – చాలా మంది ఈ వాక్యం ఎప్పుడూ చెప్పరు, మరియు అందుకే వాళ్ళు పరిగణించబడరు.
3. **సాక్ష్యం సేకరించు** (§26 brag document).
4. **Promotion cycle timing తెలుసుకో.** చాలా కంపెనీల్లో సంవత్సరానికి ఒకసారి (కొన్నిసార్లు రెండుసార్లు). Cycle కి **3 నెలల ముందు** సంభాషణ మొదలుపెట్టాలి, cycle lo కాదు.

Key Points

- Rating = **నీ పాత్రలో శ్రేష్ఠత**. Promotion = **తర్వాతి పాత్ర ప్రవర్తన**.
- Promotion ఒక బహుమతి కాదు – ఇప్పటికే జరుగుతున్నదాన్ని గుర్తించడం.
- Manager కి **స్పష్టంగా చెప్పు** నువ్వు promotion వైపు పనిచేస్తున్నావని.

- Cycle కి **3 నెలల ముందు** సంభాషణ మొదలుపెట్టు.

---

41. Certifications & internal learning platforms

వివరణ

ప్రతి పెద్ద MNC కి సొంత learning platform ఉంది – Infosys **Lex**, TCS **iON/Fresco Play**, Wipro **TopGear**, Accenture **myLearning**, Capgemini **NEXT**. వీటి గురించి ఒక సమతుల్యమైన దృష్టి:

నిజం	పరిణామం
ఇవి **నిజమైన సాంకేతిక నైపుణ్యం** ఇవ్వవు (సాధారణంగా)	బయటి interview lo ఇవి పెద్దగా సహాయపడవు
కానీ ఇవి **లోపల లెక్కలోకి వస్తాయి**	Allocation, appraisal, promotion checklist lo
కొన్ని **తప్పనిసరి**	పూర్తిచేయకపోతే rating మీద ప్రభావం
Cloud certs (AWS/Azure) **నిజంగా విలువైనవి**	ఇవి బయట కూడా గుర్తించబడతాయి

> **వ్యూహం:** తప్పనిసరి వాటిని **త్వరగా, తక్కువ శ్రమతో** ముగించు – వాటితో పోరాడొద్దు. ఆ తర్వాత నీ నిజమైన సమయాన్ని **ఒక బయటి-గుర్తింపు ఉన్న certification** కి పెట్టు (AWS Solutions Architect, Azure Fundamentals) మరియు **సొంత projects** కి.

Bench lo ఉన్నప్పుడు ఇవి ప్రత్యేకంగా ఉపయోగం

Bench కాలంలో certifications పూర్తిచేయడం రెండు పనులు చేస్తుంది: (1) allocation కి నిన్ను అర్హుడిని చేస్తుంది – చాలా projects "ఈ cert ఉన్నవాళ్ళే" అని అడుగుతాయి, (2) appraisal lo ఆ ఖాళీ కాలాన్ని పూరిస్తుంది (§2).

Key Points

- Internal platforms = **లోపలి కరెన్సీ**, బయటి విలువ తక్కువ.
- తప్పనిసరి వాటిని **త్వరగా ముగించు**; పోరాడొద్దు.
- **Cloud certifications** బయట కూడా విలువైనవి – ఇవి పెట్టుబడికి తగినవి.
- **Bench = certification సమయం.**

---

42. Onsite opportunity – నిజంగా ఎలా నిర్ణయిస్తారు

వివరణ

Onsite (client దేశంలో పనిచేయడం) చాలా మందికి ఒక పెద్ద ఆకర్షణ – ఎందుకంటే allowance వల్ల effective pay చాలా పెరుగుతుంది. కానీ ఎంపిక ఎలా జరుగుతుందో చాలామందికి తెలియదు.

నిజమైన ఎంపిక ప్రమాణాలు – ఈ క్రమంలో

1. **Client అడగడం.** చాలా onsite అవకాశాలు "client ఈ వ్యక్తిని కావాలన్నాడు" అని మొదలవుతాయి. అంటే – **client-side counterpart తో నీ సంబంధం** ఇక్కడ నిర్ణయాత్మకం (§3, §24).
2. **Grade అర్హత.** కొన్ని accounts lo కనీస grade ఉంటుంది (§9).

3. **Communication.** Client ముందు నిలబడగలిగే వాళ్ళనే పంపుతారు. ఇది technical depth కంటే ఎక్కువ బరువు.
4. **Domain/module knowledge.** ఆ నిర్దిష్ట module తెలిసినవాడు.
5. **Visa/passport సంసిద్ధత.** Passport లేకపోతే నువ్వు జాబితా నుంచి బయట. **ఇది చాలా సాధారణమైన, పూర్తిగా నివారించగలిగే అడ్డంకి.**
6. **Manager యొక్క సిఫారసు.**

> **ఇప్పుడే చేయాల్సిన ఒక పని:** passport లేకపోతే **ఈ నెలలోనే** apply చెయ్యి. Onsite అవకాశాలు తరచుగా 2–3 వారాల నోటీసుతో వస్తాయి, మరియు passport కి నెలలు పడుతుంది. "అవకాశం వచ్చాక చేసుకుంటాను" అనుకునేవాళ్ళు ఆ అవకాశాన్ని కోల్పోతారు. ఇది ఈ doc మొత్తంలో అత్యంత చౌకైన, అత్యధిక ప్రతిఫలం ఉన్న సలహా.

Onsite గురించి నిజాయితీ

- **ఆర్థికంగా** – allowance వల్ల effective savings చాలా ఎక్కువ. ఇది నిజం.
- **కెరీర్ పరంగా** – client-facing అనుభవం, మెరుగైన దృశ్యత. ఇది కూడా నిజం.
- **కానీ** – పని సాధారణంగా **coordination-heavy** అవుతుంది, coding తగ్గుతుంది. దీర్ఘకాలం onsite ఉంటే నీ సాంకేతిక పదును తగ్గే ప్రమాదం ఉంది.
- **వ్యక్తిగతంగా** – కుటుంబం, ఒంటరితనం, తిరిగి వచ్చాక సర్దుబాటు – ఇవి నిజమైన ఖర్చులు.

Key Points

- Onsite ఎంపిక: **client కోరిక > grade > communication > domain > passport > manager**.
- **Passport ఇప్పుడే తీసుకో** – ఇది అత్యంత సాధారణ, నివారించగలిగే అడ్డంకి.
- Client-side counterpart తో సంబంధం = onsite కి అతిపెద్ద కారణం.
- Onsite lo coding తగ్గి coordination పెరుగుతుంది – కళ్ళు తెరిచి వెళ్ళు.

---

43. IC track vs Lead track – ఎప్పుడు ఏది

వివరణ

4–6 సంవత్సరాల దగ్గర ఒక కూడలి వస్తుంది: **లోతుగా వెళ్ళాలా (IC – Individual Contributor), లేక వెడల్పుగా వెళ్ళాలా (Lead/Manager)?**

Service MNCs lo ఒక నిర్మాణాత్మక వాస్తవం ఉంది – **lead track ఎక్కువ స్పష్టంగా, ఎక్కువ బహుమతితో ఉంటుంది**. Architect తరహా స్వచ్ఛమైన IC పాత్రలు ఉంటాయి కానీ తక్కువ, మరియు సాధారణంగా చాలా సీనియర్ స్థాయిలో.

	IC / Technical లోతు	Lead / Delivery
నీ రోజు	Code, design, సంక్లిష్ట సమస్యలు	Planning, unblock చేయడం, client, people
నీ విజయ కొలత	సాంకేతిక నాణ్యత, సంక్లిష్టత	Team delivery, client సంతృప్తి
Service MNC lo మార్గం	Architect (తక్కువ స్థానాలు, ఆలస్యంగా)	Tech Lead → Module Lead → PM (స్పష్టం)

	IC / Technical లోతు	Lead / Delivery
Coding ఎంత మిగులుతుంది	ఎక్కువ	**వేగంగా తగ్గుతుంది**
బయటి మార్కెట్ విలువ	Product companies కి బలం	Service/consulting కి బలం

ఒక ఆచరణాత్మక సలహా: 3-5 సంవత్సరాల దగ్గర తొందరపడి నిర్ణయించుకోవద్దు. బదులుగా **"lead పనిని రుచి చూడు, కానీ hands-on వదలొద్దు"** – ఒక చిన్న module ని lead చెయ్యి, కానీ వారానికి కనీసం 50% code రాయడం కొనసాగించు. ఒకసారి పూర్తిగా coding వదిలేస్తే, తిరిగి రావడం చాలా కష్టం – ముఖ్యంగా నువ్వు తర్వాత product company కి వెళ్ళాలనుకుంటే (§45).

Key Points

- Service MNC lo **lead track ఎక్కువ స్పష్టం**; IC/architect మార్గం ఇరుకు.
- 3-5 సంవత్సరాల దగ్గర **నిర్ణయించాల్సిన అవసరం లేదు** – రెండిటినీ రుచి చూడు.
- **Coding పూర్తిగా వదలొద్దు** – తిరిగి రావడం కష్టం.

---

44. ఎప్పుడు మారాలి – project change vs company change

వివరణ

అసంతృప్తి వచ్చినప్పుడు చాలా మంది వెంటనే "కంపెనీ మారాలి" అనుకుంటారు. కానీ చాలా సందర్భాల్లో సమస్య **కంపెనీ కాదు, project**. మరియు project మార్చడం చాలా చౌక.

అసంతృప్తి · మూలం ఎక్కడ, పరిష్కారం ఏది

మొదట సమస్యని సరిగ్గా గుర్తించు

ఇవి PROJECT సమస్యలు

విసుగు పుట్టించే పని, support-only project
పాత stack, నేర్చుకునేది లేదు
కష్టమైన client / project manager
భయంకరమైన timezone
→ పరిష్కారం: internal project change

ఇవి COMPANY సమస్యలు

Pay మార్కెట్ కంటే చాలా వెనుక
రెండుసార్లు promotion తిరస్కరణ
మొత్తం కంపెనీలో నీ stack కి భవిష్యత్తు లేదు
విశ్వాసం పోయింది (layoffs, జీతం ఆలస్యం)
→ పరిష్కారం: switch

Internal project change – ఎలా అడగాలి

1 · ముందు **reporting manager** తో మాట్లాడు – RMG కి నేరుగా వెళ్ళొద్దు (అది అతనికి అగౌరవం)
2 · కారణాన్ని **ముందుకు** చూపిస్తూ చెప్పు: "నేను React lo లోతుగా వెళ్ళాలనుకుంటున్నాను" – "ఈ project విసుగ్గా ఉంది" అని కాదు
3 · ప్రస్తుత project ని **సరిగ్గా handover** చేస్తానని హామీ ఇవ్వు – ఇదే అతని అసలు ఆందోళన
4 · సమయం ఇవ్వు – release తర్వాత అని చెప్పు. తొందరపెడితే "సహకరించనివాడు" అనే ముద్ర పడుతుంది

Switch కి సరైన సమయం

- **కనీసం 2 సంవత్సరాలు** ఒక కంపెనీలో (§7) – తక్కువైతే resume దెబ్బతింటుంది.
- **Appraisal తర్వాత**, ముందు కాదు – hike మరియు rating తీసుకున్నాకే వెళ్ళు.
- **ఒక పూర్తి project cycle** ముగిశాక – ఇది నీకు ఒక పూర్తి కథ ఇస్తుంది (§37).
- **Notice period** ని లెక్కలోకి తీసుకో – 90 రోజులంటే నువ్వు ఇప్పుడే మొదలుపెట్టాలి.

Key Points

- చాలా అసంతృప్తి **project సమస్య**, company సమస్య కాదు – మొదట సరిగ్గా గుర్తించు.
- Internal change: **manager ముందు**, ముందుకు చూసే కారణం, **handover హామీ**, సమయం ఇవ్వడం.
- Switch: **2+ సంవత్సరాలు, appraisal తర్వాత, ఒక పూర్తి cycle ముగిశాక**.

---

45. Exit path – services నుంచి GCC / product కి

వివరణ

చాలా మంది service MNC lo 2-4 సంవత్సరాలు చేసి, ఆ తర్వాత **GCC** (global capability centre – Target, Walmart Global Tech, Lowe's, JPMC, Goldman, ThoughtWorks తరహా) లేదా product company కి వెళ్ళాలనుకుంటారు. ఇది పూర్తిగా సాధ్యం – కానీ **అంతరం నిజమైనది**, మరియు దాన్ని ఉద్దేశపూర్వకంగా పూరించాలి.

అంతరం ఎక్కడ ఉంది

GCC/product interview lo అడిగేది	Service MNC అనుభవం ఇచ్చేది	అంతరం
DSA (LeetCode Medium)	దాదాపు ఏమీ లేదు	**అతిపెద్ద అంతరం**
Frontend system design లోతుగా	కొంత, కానీ మీరు నిర్ణయించలేదు	మధ్యస్థం
"నువ్వు ఏ నిర్ణయం తీసుకున్నావు?"	Stack client నిర్ణయించాడు (§19)	మధ్యస్థం
Product ఆలోచన – "ఈ feature ఎందుకు?"	సాధారణంగా అడగలేదు	మధ్యస్థం
Ownership, delivery క్రమశిక్షణ	**చాలా బలం**	అంతరం లేదు – **ఇది నీ ఆయుధం**
Communication, client-facing	**చాలా బలం**	అంతరం లేదు

ఆరు నెలల ప్రణాళిక – services నుంచి GCC కి

1. **DSA** – వారానికి 4-5 problems, 6 నెలలు. LeetCode Top Interview 150 సరిపోతుంది. (`DSA_Patterns_Telugu.md` మరియు `DSA_01..10` చూడు.)
2. **ఒక సొంత project** – నీవే నిర్ణయాలు తీసుకున్నది. Deployed, GitHub lo, README తో. ఇది "నువ్వు ఏ నిర్ణయం తీసుకున్నావు?" ప్రశ్నకి సమాధానం.
3. **Frontend system design** – §36 చట్రం + `HLD_Telugu.md` నుంచి caching, CDN, API design.
4. **నీ కథలని మళ్ళీ రాయి** – service context నుంచి *నిర్ణయాలు మరియు ప్రభావం మీదికి*. "Client ఇలా అడిగాడు" కాదు, "నేను ఇలా ప్రతిపాదించాను, ఎందుకంటే..."
5. **సొంత GitHub** నిర్వహించు (§19) – client code చూపించలేవు.

ఒక ముఖ్యమైన మానసిక మార్పు: service అనుభవాన్ని **క్షమాపణగా** చెప్పొద్దు. "నేను కేవలం service company lo ఉన్నాను" – ఇది నిన్ను నువ్వే తగ్గించుకోవడం. బదులుగా: **"నేను ఒక పెద్ద, పాత codebase lo, పరిమితుల్లో, నిజమైన users కోసం, ప్రక్రియని గౌరవిస్తూ deliver చేశాను"** – ఇది చాలా product company engineers కి **లేని** అనుభవం, మరియు hiring managers దీన్ని విలువైనదిగా చూస్తారు.

Key Points

- అతిపెద్ద అంతరం **DSA** – 6 నెలల స్థిరమైన ప్రయత్నం కావాలి.
- రెండో అంతరం: **"నీ నిర్ణయం ఏమిటి?"** – దీనికి సొంత project సమాధానం.
- నీ బలాలు: **ownership, delivery క్రమశిక్షణ, communication, పెద్ద codebase అనుభవం**.
- Service అనుభవాన్ని **క్షమాపణగా కాదు, ఒక ప్రత్యేకతగా** చెప్పు.

Part 6 – Red Flags & నిజాలు

46. Requisition / project lo red flags

వివరణ

ప్రతి offer మంచి offer కాదు. Interview సమయంలోనే గుర్తించగలిగే హెచ్చరిక సంకేతాలు:

Red flag	ఏమి సూచిస్తుంది	ఎలా కనిపెట్టాలి
JD చాలా విస్తృతం ("React, Angular, Node, Java, DevOps")	Role స్పష్టత లేదు; నిన్ను ఎక్కడైనా పెడతారు	"What will I be doing in the first three months?"
"Support and maintenance" అనే పదాలు	కొత్త development తక్కువ (§47)	"What's the split between new features and defects?"
Interviewer project గురించి చెప్పలేకపోవడం	Req ఒక general pipeline కోసం (§27)	"Which account is this for?"
చాలా వేగంగా offer (ఒకే round తర్వాత)	Attrition ఎక్కువ కావచ్చు	"Why is this position open?"
Rotational shifts / 24×7 support	Work-life ప్రభావం	నేరుగా అడుగు – ఇది offer letter lo ఉండాలి
"Flexible with technology"	నీ React skill వాడకపోవచ్చు	"What percentage of the role is React?"
Grade చెప్పడానికి తటపటాయించడం	Band తక్కువ కావచ్చు	§6
Notice period buy-out గురించి అస్పష్టత	తర్వాత సమస్య	రాతపూర్వకంగా అడుగు

> **నియమం:** ఈ ప్రశ్నలు అడగడం దురుసుతనం కాదు. మంచి కంపెనీలు వీటికి స్పష్టంగా సమాధానం ఇస్తాయి. **సమాధానం దాటవేయడమే** అసలు red flag.

Key Points

- అతి విస్తృతమైన JD = role స్పష్టత లేదు.
- "Support and maintenance" అనే పదాలని గమనించు.
- **"Why is this position open?"** – ఒక చిన్న ప్రశ్న, చాలా సమాచారం.
- ప్రశ్నలని దాటవేయడమే అసలు హెచ్చరిక.

---

47. Support project vs development project

వివరణ

ఇది నీ career వేగాన్ని అత్యధికంగా ప్రభావితం చేసే ఒక్క అంశం – మరియు చాలా మంది దీన్ని join అయ్యాకే తెలుసుకుంటారు.

	Development project	Support / maintenance project
రోజువారీ పని	కొత్త features, design చర్చలు	Defects, incidents, చిన్న మార్పులు
Code lo నీ భాగం	కొత్తది రాయడం	ఉన్నదాన్ని మార్చడం
నేర్చుకునేది	Architecture, కొత్త నమూనాలు	Debugging, పాత systems, production
Interview కథలు	సులభంగా వస్తాయి	కష్టం – "నేను bugs fix చేశాను" బలహీనం
Resume మీద ప్రభావం	బలం	2+ సంవత్సరాలు ఉంటే **ప్రమాదం**
ఒత్తిడి	Sprint deadlines	**On-call, SLA, రాత్రి calls**

ఇది "support చెడ్డది" అని కాదు. ఒక సంవత్సరం support lo నేర్చుకునే production debugging, log reading, ఒత్తిడిలో ప్రశాంతత – ఇవి నిజమైన, అరుదైన నైపుణ్యాలు (§25). సమస్య **ఎక్కువ కాలం** అక్కడే ఉండటం: రెండేళ్ళ తర్వాత నీ resume lo "నేను ఏమి కట్టాను?" అనే ప్రశ్నకి సమాధానం ఉండదు, మరియు అదే interview lo అడుగుతారు.

Support project lo ఉంటే – ఏమి చేయాలి

1. **Enhancement tickets ని వెతుకు.** పూర్తిగా support projects lo కూడా చిన్న enhancements ఉంటాయి. వాటిని కావాలని తీసుకో.
2. **Automation కట్టు.** పునరావృత్త manual పనిని script చెయ్యి – ఒక dashboard, ఒక monitoring alert, ఒక log parser. **ఇది "నేను కట్టాను" అనే కథ అవుతుంది.**
3. **RCA లని కథలుగా మార్చు** (§25) – "P1 incident, root cause, 40 నిమిషాల్లో workaround" ఒక బలమైన కథ.
4. **12-18 నెలల్లో మార్పు అడుగు** (§44).

Key Points

- Support project = debugging నేర్చుకుంటావు, కానీ **"నేను కట్టాను" కథలు రావు**.
- **12-18 నెలలు సరిపోతుంది**; 2+ సంవత్సరాలు resume ప్రమాదం.
- అక్కడ ఉంటే: **enhancements + automation + RCA కథలు**.

48. Stack stagnation – ఎలా తప్పించుకోవాలి

వివరణ

Service company lo stack ని client నిర్ణయిస్తాడు (§11). అంటే నువ్వు 3 సంవత్సరాలు AngularJS 1.x మీద కూర్చోవచ్చు – మరియు బయటి మార్కెట్ నిన్ను వదిలేసి ముందుకు వెళ్ళిపోవచ్చు. ఇది ఈ కెరీర్ మార్గంలో **అతిపెద్ద నిజమైన ప్రమాదం**.

మూడు రక్షణలు

1. ప్రతి 6 నెలలకి ఒక "మార్కెట్ తనిఖీ". Naukri lo నీ profile తో ఎన్ని relevant calls వస్తున్నాయి? ఒక్క interview ఇచ్చి చూడు – offer తీసుకోవాల్సిన అవసరం లేదు. ఇది నీ మార్కెట్ విలువకి ఒక ఉచిత కొలత.

2. ఒక సజీవ side project. వారానికి 3–4 గంటలు చాలు. ముఖ్యం ఏమిటంటే ఇది **నీ stack ని తాజాగా ఉంచుతుంది** – React 19, TypeScript, testing, deployment. మరియు ఇది నీది, client కి చెందినది కాదు (§19).

3. ప్రతి సంవత్సరం ఒక కొత్త స్తంభం. ఒక సంవత్సరం TypeScript లోతుగా. తర్వాతి సంవత్సరం testing. తర్వాతిది accessibility లేదా performance. ఇది frameworks ని వెంటాడటం కాదు – **మారని పునాదులని** బలపరచడం.

ఏవి త్వరగా పాతబడతాయి, ఏవి పాతబడవు

నీ సమయాన్ని ఎక్కడ పెట్టాలి – ఒక సరళమైన నియమం

వేగంగా పాతబడేవి	పాతబడనివి
ఒక నిర్దిష్ట state library యొక్క API	JavaScript భాష, browser ఎలా పనిచేస్తుంది
Build tool config (Webpack → Vite)	HTTP, caching, network
CSS-in-JS యొక్క ఈ నెల ఫ్యాషన్	Accessibility, semantics
Framework-నిర్దిష్ట ఉపాయాలు	Debugging, testing, code చదవడం
→ అవసరమైనప్పుడు నేర్చుకో, ముందు కాదు	**→ ఇక్కడ పెట్టుబడి పెట్టు**

2026 lo ఒక ముఖ్యమైన మార్పు

Component స్థాయి పని – "ఒక button component రాయి" – AI తో **వేగంగా commodity అవుతోంది.**
విలువ పెరుగుతున్నది: design tokens, accessibility, performance, micro-frontends,
పెద్ద codebase refactors – అంటే **system స్థాయి పని**.
ఇది §49 lo విస్తరిస్తాం – కానీ దిశ స్పష్టం: ఒక్కో component కాదు, **మొత్తం వ్యవస్థ** గురించి ఆలోచించడం నేర్చుకో.

Key Points

- Stack stagnation = **ఈ కెరీర్ మార్గంలో అతిపెద్ద నిజమైన ప్రమాదం**.
- మూడు రక్షణలు: **6-నెలల మార్కెట్ తనిఖీ, సజీవ side project, సంవత్సరానికి ఒక పునాది**.
- **పాతబడనివాటిలో పెట్టుబడి పెట్టు** – భాష, browser, HTTP, a11y, debugging.
- దిశ: component స్థాయి నుంచి **system స్థాయికి**.

---

49. AI shift – ఏది commodity అయ్యింది, ఏది విలువ పెరిగింది

వివరణ

2026 lo ఇది ఒక వాస్తవం, ఒక అంచనా కాదు: **ఒక component రాయడం చౌకైపోయింది.** AI సాధనాలు ఒక form, ఒక table, ఒక modal ని నిమిషాల్లో ఇస్తాయి. ఇది frontend engineer పాత్రని మారుస్తోంది – దాన్ని తొలగించట్లేదు, కానీ **విలువ ఎక్కడ ఉందో మారుస్తోంది**.

చొక్కైపోయినది	ఖరీదైనదిగా మిగిలినది
ఒక component యొక్క మొదటి draft రాయడం	**ఆ component ఏమి ఉండాలో నిర్ణయించడం**
Boilerplate, config, test scaffolding	సరైన **abstraction** ఎంచుకోవడం
Syntax, API గుర్తుంచుకోవడం	ఒక పెద్ద codebase lo **ఎక్కడ మార్చాలో** తెలియడం
ఒక తెలిసిన bug ని fix చేయడం	**ఎందుకు జరిగిందో** అర్థం చేసుకోవడం, మళ్ళీ జరగకుండా ఆపడం
Documentation యొక్క మొదటి draft	Requirement lo **ఖాళీని కనిపెట్టడం** (§21)

ఆచరణలో ఏమి మారాలి

1. **AI ని వాడు – దాచొద్దు.** చాలా MNCs ఇప్పుడు approved AI సాధనాలు ఇస్తున్నాయి. వాడకపోవడం ఒక ప్రతికూలత. (కానీ **client code ని approve కాని సాధనాల్లో పెట్టొద్దు** – ఇది తీవ్రమైన IP ఉల్లంఘన.)
2. **Review నైపుణ్యం పెంచుకో.** AI ఇచ్చిన code ని *ఎందుకు తప్పో* చెప్పగలగడం – ఇది ఇప్పుడు ఒక ప్రధాన నైపుణ్యం. దీనికి పునాదులు కావాలి (§48).
3. **System స్థాయికి ఎదుగు.** Design tokens, accessibility, performance బడ్జెట్లు, micro-frontend సరిహద్దులు, పెద్ద refactors – ఇవి సందర్భాన్ని కోరతాయి, మరియు అదే AI దగ్గర లేనిది.
4. **Interview lo దీని గురించి నమ్మకంగా మాట్లాడు.** "AI వాడతారా?" అనే ప్రశ్న ఇప్పుడు సాధారణం.

"Do you use AI tools in your work?" కి ఒక మంచి సమాధానం:

"Yes, for the parts where it's genuinely faster – scaffolding a component, writing the first pass of tests, converting a design into markup. What I don't do is accept it without reading it. The two places it's burned us are accessibility, where it produces div-with-onClick quite happily, and effect dependencies, where it writes something that looks right and re-renders in a loop. So I treat it like a fast junior developer: useful, quick, and everything still goes through the same review. The thinking it hasn't replaced is deciding what the component's API should be in the first place – that needs context about the other forty screens."

Key Points

- **Component స్థాయి పని commodity అవుతోంది**; system స్థాయి పని విలువ పెరుగుతోంది.
- AI ని **వాడు, కానీ approve కాని సాధనాల్లో client code పెట్టొద్దు**.
- కొత్త ప్రధాన నైపుణ్యం: **AI output ని విమర్శనాత్మకంగా review చేయడం** – దీనికి పునాదులు కావాలి.
- Interview lo దీని గురించి **నమ్మకంగా, నిర్దిష్టంగా** మాట్లాడు.

ముగింపు – 30 రోజుల ప్రణాళిక

49 sections చదవడం ఒక పని; దానితో ఏదైనా చేయడం వేరే పని. నువ్వు **ఇప్పుడు ఎక్కడ ఉన్నావో** దాన్ని బట్టి ఒక ప్రణాళిక.

30 రోజులు · నీ పరిస్థితిని బట్టి

మూడు పరిస్థితులు, మూడు ప్రణాళికలు

నువ్వు ఇప్పుడు job వెతుకుతున్నావు

వారం 1 · Naukri profile keywords (§28) · 6 కథలు రాయి (§37) · passport తనిఖీ (§42)
వారం 2 · JavaScript bank (§32) – రోజుకి 5 ప్రశ్నలు, ప్రతిదానికీ ఒక నిజమైన ఉదాహరణతో
వారం 3 · React bank (§33) + ఒక చిన్న coding task రోజూ (§35)
వారం 4 · 4 కంపెనీలకి **ఒకేసారి** apply (§27 – పోటీ offer కోసం) · mock interview

నువ్వు ఇప్పటికే ఒక MNC lo ఉన్నావు

వారం 1 · **Brag document** మొదలుపెట్టు (§26) – గత 6 నెలలు వెనక్కి వెళ్ళి నింపు
వారం 2 · Manager తో 1:1 – "తర్వాతి grade కి నిర్దిష్టంగా ఏమి కావాలి?" (§40)
వారం 3 · నీ project support నా development నా అని నిజాయితీగా అంచనా వెయ్యి (§47)
వారం 4 · ఒక side project మొదలుపెట్టు (§48) · passport (§42) · ఒక cert ఎంచుకో (§41)

నువ్వు GCC / product కి వెళ్ళాలనుకుంటున్నావు

వారం 1 · DSA మొదలుపెట్టు – వారానికి 4-5 problems, ఇది 6 నెలల పని (§45)
వారం 2 · నీ కథలని మళ్ళీ రాయి – "client అడిగాడు" నుంచి "నేను ప్రతిపాదించాను" కి
వారం 3 · సొంత project – నువ్వే నిర్ణయాలు తీసుకున్నది, deployed, README తో
వారం 4 · Frontend system design (§36) + HLD పునాదులు

ఈ doc నుంచి పది వాక్యాలు మాత్రమే గుర్తుంచుకుంటే

1. Service company **మనుషుల సమయాన్ని margin తో అమ్ముతుంది** – మిగతాదంతా దీని నుంచే వస్తుంది (§1).
2. **Billable = సురక్షితం.** Bench lo చోరవ తీసుకున్నవాడు త్వరగా బయటపడతాడు (§2).
3. నీ **Reporting Manager ≠ Project Manager** – నీ పని ఇద్దరికీ కనిపించాలి (§3).
4. **Code రాయడం పని lo ~40%.** కనిపించని పని = జరగని పని (§4).
5. Offer lo నీ grade ని అత్యధికంగా నిర్ణయించేది **నీ ప్రస్తుత CTC** – దాన్ని విరగ్గొట్టేది **ఇంకో offer** (§6, §38).
6. **CTC కాదు, fixed pay** పోల్చు (§10).
7. Technical round lo reject కి #1 కారణం **vanilla JavaScript**, React కాదు (§31, §32).
8. **Managerial round lo ఎక్కువ rejections** జరుగుతాయి – ఆరు కథలు సిద్ధం చేసుకో (§37).
9. **Rating ≠ promotion.** Promotion అంటే ఇప్పటికే తర్వాతి పాత్ర చేస్తున్నావని గుర్తించడం (§40).
10. **Passport ఇప్పుడే తీసుకో.** ఇది ఈ doc lo అత్యంత చౌకైన, అత్యధిక ప్రతిఫలం ఉన్న సలహా (§42).

ఈ library lo తర్వాత ఎక్కడికి

నీకు కావలసినది	ఏ doc
Infosys interview rounds, ప్రశ్నలు	`Infosys_Interview_Prep_Telugu.md`
React లోతు (63 topics, Fiber నుంచి RSC వరకు)	`React_Telugu.md`
JavaScript పూర్తి భాష	`JavaScript_Telugu.md`
TypeScript	`TypeScript_Telugu.md`
DSA – pattern-first	`DSA_Patterns_Telugu.md`, తర్వాత `DSA_01..10`
HLD / LLD భావనలు	`HLD_Telugu.md`, `LLD_Telugu.md`

నకు కావలసినది	ఏ doc
Design interview problems	`HLD_Design_Problems_Telugu.md`, `LLD_Design_Problems_Telugu.md`
CS పునాదులు	`OS_`, `CN_`, `DBMS_`, `Security_Telugu.md`

చివరిగా, ఒక నిజాయితీ మాట. ఈ doc lo చాలా భాగం – grades, margins, bell curves, billability – మొదట చదివితే నిరుత్సాహంగా అనిపించొచ్చు. కానీ ఈ వ్యవస్థని **స్పష్టంగా చూడటమే** అందులో బాగా పనిచేయడానికి మొదటి అడుగు. చాలా మంది ఈ నియమాలు తెలియకుండా ఐదేళ్ళు పనిచేసి, ఆ తర్వాత "ఎందుకు నాకు promotion రాలేదు" అని ఆశ్చర్యపోతారు. నువ్వు ఇప్పుడే వాటిని తెలుసుకున్నావు.

మరియు ఒక విషయం మర్చిపోవద్దు: ఒక పెద్ద MNC lo, ఒక నిజమైన client కోసం, పరిమితుల్లో, ఒక పాత codebase lo, ఒక జట్టుతో కలిసి deliver చేయడం – **ఇది నిజమైన engineering**. దీన్ని తక్కువ చేసి చూడాల్సిన అవసరం ఎప్పుడూ లేదు.